



UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Desarrollo de un sistema web colaborativo para publicación y gestión de eventos abiertos

**Diseño e Implementación de un Sistema web abierto para usuarios para la creación y gestión de
eventos en toda España**

Autor

Francisco Quiles Ramírez

Director

Juan Carlos Torres Cantero



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicaciones

Granada, Julio 2025

Desarrollo de un sistema web colaborativo para publicación y gestión de eventos abiertos

Francisco Quiles Ramírez

Palabras clave: Eventos, Plataforma, Autogestión, Spring Boot, React, Usuario, Inscrito.

Resumen

Este Trabajo de Fin de Grado presenta el desarrollo de una plataforma web intuitiva para descubrir y publicar eventos, motivada por la escasez de plataformas locales que permiten la autogestión de eventos, para conectar a personas con intereses comunes sin depender de redes sociales. La plataforma estará disponible para toda España, incorporando un buscador geográfico que permite seleccionar de qué zona quieres buscar los eventos. Además los usuarios podrán aplicar unos filtros de categoría, precio, fecha sobre la ciudad seleccionada anteriormente o en general.

El acceso a la plataforma está abierto tanto a usuarios invitados (sin registro) que simplemente vean la página con los eventos disponibles o usuarios que se registren y puedan apuntarse a los eventos o crearlos, para ello en la parte superior derecha habrá una opción de iniciar sesión o registrarse, cualquier usuario registrado podrá crear un evento, además también tendrán la opción de valorar los eventos terminados y dejar un comentario y así servir de ayuda a los usuarios nuevos que lleguen. El registro cumple con la ley de protección de datos y requiere un correo, la contraseña y un nombre de usuario que se recogerán en una base de datos, para poder iniciar sesión de forma fácil y segura.

Respecto a los eventos, aparecerán todos los que haya en el filtro de búsqueda de la ciudad que hayamos elegido, y habrá una columna a la izquierda con otros filtros para búsquedas más concretas como puede ser la fecha o categoría donde aparecerán unas categorías ya prefijadas, incluso si queremos eventos gratis o con un precio máximo.

Los usuarios registrados tendrán una sección personalizada donde podrán visualizar los eventos a los que se han inscrito con la posibilidad de desapuntarse antes de que empiecen o de dejar una valoración y un comentario si el evento ya ha finalizado, esta valoración se le asignará al usuario creador el cual contendrá una valoración media a través de las valoraciones medias de sus eventos creados. El creador

de un evento podrá en todo momento modificar y eliminar sus eventos creados, además de ver la lista de usuarios inscritos a él, y una vez finalizado ver la valoración media de su evento y los comentarios de los participantes. De esta manera se promueve una comunidad activa y transparente en torno a los eventos creados por los propios usuarios.

Development of a collaborative web system for publishing and managing open events

Francisco Quiles Ramírez

Keywords: Event, Platform, Self-management, Spring Boot, React, User, Registered.

Abstract

This Final Degree Project presents the development of an intuitive web platform for discovering and publishing events, motivated by the scarcity of local platforms that allow for self-management of events, in order to connect people with common interests without relying on social networks. The platform will be available throughout Spain, incorporating a geographical search engine that allows users to select the area in which they want to search for events. In addition, users will be able to apply filters for category, price, and date to the previously selected city or in general.

Access to the platform is open to both guest users (without registration) who simply view the page with the available events, and users who register and can sign up for events or create them. To do so, there will be an option to log in or register in the upper right corner. Any registered user can create an event, and they will also have the option to rate completed events and leave a comment, thus helping new users who arrive. Registration complies with data protection law and requires an email address, password and username, which will be stored in a database to enable easy and secure login.

With regard to events, all those in the search filter for the city we have chosen will appear, and there will be a column on the left with other filters for more specific searches, such as date or category, where pre-set categories will appear, including whether we want free events or events with a maximum price.

Registered users will have a personalised section where they can view the events they have signed up for, with the option to unsubscribe before they start or leave a rating and comment if the event has already ended. This rating will be assigned to the user who created the event, who will receive an average rating based on the average ratings of their created events. The creator of an event can modify and delete their created events at any time, as well as view the list of users registered for it, and once it is over, view the average rating of their event and the comments of the participants. This promotes an active and transparent community around the events created by the users themselves.

Yo, **Francisco Quiles Ramírez**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 26583235L, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Francisco Quiles Ramírez

A handwritten signature in black ink, appearing to read 'F. Quiles', with a stylized flourish underneath.

Granada a 30 de Julio de 2025

D. Juan Carlos Torres Cantero, Profesor del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado Desarrollo de un sistema web colaborativo para publicación y gestión de eventos abiertos, ha sido realizado bajo su supervisión por Francisco Quiles Ramírez, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 4 de Septiembre de 2025.

Firmado por TORRES CANTERO JUAN CARLOS -
***6419** el día 04/09/2025 con un certificado
emitido por AC FNMT Usuarios

Los directores:

Juan Carlos Torres Cantero

Agradecimientos

No habría sido posible llegar a este momento donde finaliza mi carrera universitaria sin el apoyo de mi familia, amigos y compañeros que me han acompañado en este camino, y a todos los profesores que me han impartido clases y me han ayudado durante estos cuatro años.

Gracias.

ÍNDICE

Resumen.....	3
Abstract.....	5
Punto 1: Introducción.....	17
1.1 Motivación.....	17
1.2 Objetivos.....	18
1.3 Estructura proyecto.....	18
Punto 2: Estado del arte.....	20
2.1 Historia de los eventos.....	20
2.2 Plataformas de gestión de eventos en la actualidad.....	21
2.3 Herramientas actuales en desarrollo web de eventos.....	21
Punto 3: Planificación del proyecto.....	22
3.1 Fase inicial.....	22
3.2 Fase de análisis.....	23
3.3 Fase de desarrollo.....	23
3.4 Fase de prueba y correcciones.....	24
3.5 Elaboración de la memoria.....	25
Punto 4: Análisis de funcionalidad de la página.....	26
4.1 Objetivo funcional de la plataforma.....	26
4.2 Usuarios.....	26
4.3 Análisis de requisitos.....	27
4.3.1 Requisitos funcionales.....	27
4.3.2 Requisitos no funcionales.....	28
4.4 Modelos de casos de uso y escenarios de uso.....	29
4.5 Ley de protección de datos.....	30
4.6 Análisis de tecnologías.....	30
4.6.1 Frontend.....	30
4.6.2 Backend.....	32
4.6.3 Base de datos.....	32
4.6.4 Servidor.....	33
Punto 5: Diseño.....	34
5.1 Modelo y arquitectura del sistema.....	34
5.2 Diseño de la base de datos.....	35
5.3 Diseño de las interfaces.....	38
Punto 6: Implementación.....	42
6.1 Backend.....	42
6.2 Frontend.....	46
6.2.1 Registro.....	47
6.2.2 Login.....	48

6.2.3 Recuperar contraseña.....	49
6.2.4 Crear evento.....	51
6.2.5 HomePage (Búsquedas y eventos).....	52
6.2.6 Mis eventos.....	54
6.3 Servidor.....	58
Punto 7: Pruebas y resultados.....	61
7.1 Registro.....	61
7.2 Inicio de sesión.....	62
7.3 Cambiar contraseña.....	63
7.4 Crear eventos.....	64
7.5 Visualizar eventos.....	65
7.6 Inscribirse.....	66
7.7 Filtros y buscador.....	67
7.8 Gestión de eventos creados.....	69
7.9 Gestión de eventos inscrito.....	70
7.10 Valoración de eventos.....	71
7.11 Comentar eventos.....	71
Despliegue en servidor.....	72
Punto 8: Conclusiones y trabajo futuro.....	73
8.1 Conclusiones.....	73
8.2 Trabajo Futuro.....	74
Bibliografía.....	75

ÍNDICE DE FIGURAS

Ilustración 1: Diagrama de casos de uso del sistema
Ilustración 2: Capas del sistema
Ilustración 3: Tablas del sistema
Ilustración 4: Mockup Página Principal
Ilustración 5: Mockup Página Principal con sesión iniciada
Ilustración 6: Mockup Evento en Página Principal
Ilustración 7: Mockup Detalles Evento
Ilustración 8: Mockup Menú Usuarios
Ilustración 9: Mockup Página Mis Eventos
Ilustración 10: Estructura Backend
Ilustración 11: Estructura Frontend

Ilustración 12: Servicios creados en Render

Ilustración 13: Página de Registro

Ilustración 14: Header tras Iniciar Sesión

Ilustración 15: Página de Iniciar Sesión

Ilustración 16: Menú de opciones de usuario

Ilustración 17: Menú para introducir código para el cambio de contraseña

Ilustración 18: Menú cambio de contraseña

Ilustración 19: Creación de Evento

Ilustración 20: Información Evento

Ilustración 21: Mensaje en Eventos si no estás registrado

Ilustración 22: Mensaje evento completo

Ilustración 23: Mensaje tras Inscribirse a un Evento

Ilustración 24: Buscador de Ciudad en el Header

Ilustración 25: Resultados Búsqueda Eventos en Granada

Ilustración 26: Filtros de Búsqueda Menú Izquierdo

Ilustración 27: Mis Eventos Creados

Ilustración 28: Mis Eventos Inscrito

Ilustración 29: Comentar Evento

Ilustración 30: Evento Comentado

INDICE DE CÓDIGO

Código 1: Clase Usuario

Código 2: Métodos Repositorio Usuario

Código 3: Funcionalidades de Registro y Login de Usuarios

Código 4: Funcionalidades de Crear y Listar Eventos

Código 5: Conexión a la base de datos

Código 6: Registro de Usuarios en el Frontend

Código 7: Login de Usuarios en el Frontend

Código 8: Envío y Validación de código para recuperar contraseña

Código 9: Cambio de contraseña

Código 10 Creación de Eventos en el Frontend

Código 11: Detalle de los Eventos en el Frontend

Código 12: Desapuntarse y Valora Evento en el Frontend

Código 13: Comentar Eventos en el Frontend

Código 14: Ver Eventos Creados e Inscrito en el Frontend

Código 15: Eliminar Evento y ver Inscritos en el Frontend

Código 16: Dockerfile para conectar con Render

Código 17: URL para detectar Backend

Código 18: Variable con enlace a Backend

Código 19: Conexión Backend y Frontend en entorno local

Código 20: Conexión Backend y Frontend en el servidor

Punto 1: Introducción

En la actualidad, cada vez son más las personas interesadas en participar en actividades sociales, culturales o deportivas tanto como para conocer gente nueva o disfrutar de su tiempo libre, sin embargo la mayoría de plataformas dependen de redes sociales o son aplicaciones centralizadas que no es sencillo o accesible para todos los usuarios. Esta carencia motiva a la creación de este Trabajo de Fin de Grado que consiste en una página interactiva para los usuarios sobre eventos en toda España con diferentes filtros según los gustos del usuario, permitiendo tanto inscribirse como crear sus propios eventos si estás registrado, ofreciendo una solución práctica y escalable para facilitar la organización de eventos, fomentar la participación local y mejorar la accesibilidad a la información de actividades de forma descentralizada y segura.

La página estará desarrollada como un sistema web moderno, combinando un frontend React donde se utiliza HTML y CSS y un backend Java con Spring Boot, conectada a una base de datos relacional PostgreSQL, y finalmente desplegada en un servidor como Render.

1.1 Motivación

En la actualidad la mayoría de eventos se organiza a través de redes sociales o plataformas cerradas, donde tanto la visibilidad como el filtrado de intereses es limitado. Hoy día existen muchas personas interesadas en asistir a eventos y actividades deportivas, culturales o sociales pero no las encuentran fácilmente en su zona, al igual que otras pretenden organizarlos pero no disponen de los suficientes medios y herramientas para llevarlo a cabo.

La idea de este proyecto surge de esa necesidad real de crear una plataforma abierta, sencilla y funcional donde cualquier persona, sin conocimientos técnicos, pueda publicar un evento y así llegar a usuarios interesados, fomentado la participación, la conectividad local y el acceso libre a la información de actividades y así unir gente con los mismos gustos, intereses y que quizás no consiguen relacionarse con facilidad.

Además, este proyecto presenta una gran oportunidad de aplicar conocimientos adquiridos a lo largo del grado de Ingeniería Informática, en especial a la mención de Sistemas de Información que he realizado y poner en práctica mi conocimiento sobre desarrollo web frontend y backend, bases de datos, diseño de interfaces y despliegue de un servidor, dentro de una aplicación práctica y con potencial de uso real.

1.2 Objetivos

El propósito central del proyecto es desarrollar una aplicación web que permita tanto descubrir como publicar eventos de forma autogestionada, intuitiva y accesible. Entre los objetivos específicos destacan:

- Diseñar y construir una interfaz web clara y fácil de usar que permita filtrar eventos por ciudad, categoría, precio y fecha.
- Permitir al usuario consultar información detallada de cada evento como el nombre, lugar, capacidad, precio, valoración, descripción, creador.
- Asegurar la protección de datos personales cumpliendo con las normativa(mínimo de datos, sin acceso público)
- Desarrollar una API REST con Spring Boot que permite que conecte con una base de datos relacional PostgreSQL
- Desplegar el proyecto en un servidor web como Render, permitiendo su acceso desde Internet.
- Aplicar de forma práctica los conocimientos adquiridos durante el grado.

1.3 Estructura proyecto

Para facilitar la navegación del proyecto vamos a comentar su estructura, lo primero será el **resumen** donde se proporciona una síntesis concisa de lo que tratara y el funcionamiento del mismo. Tras esto podremos ver el Índice completo de la Memoria y unos índices adicionales de las imágenes y de los códigos que aparecen durante el proyecto.

En este **primer capítulo** se ha hecho una introducción del proyecto y se ha comentado la motivación que ha permitido llevarlo a cabo, además de mencionar el objetivo principal y los objetivos específicos.

En el **punto dos** se tratará el estado del arte del proyecto donde podremos ver el origen y la evolución de la gestión de eventos hasta en la actualidad, se comentarán páginas de gestión de eventos actuales, así como las tecnologías más comunes empleadas.

El **tercer apartado** trata de la planificación del proyecto donde se ha dividido en una fase inicial donde aparecen las primeras ideas y propuestas, una fase de análisis en la que se dejó clara la funcionalidad, requisitos y herramientas, una fase de desarrollo donde se puso todo esto en práctica y finalmente una fase de prueba y correcciones para verificar que todo funcione correctamente. Además se ha añadido un apartado de elaboración de la memoria donde se desarrolla el proceso de redacción de este documento.

En el punto **número cuatro** se realiza un análisis más profundo del proyecto y del funcionamiento de la página, así como el objetivo funcional de la plataforma y los requisitos funcionales y no funcionales, se comentan los casos de uso y escenarios de uso y un análisis de las tecnologías empleadas.

En el **quinto apartado** se habla del diseño del proyecto, donde al principio aparecerá el modelo y arquitectura del sistema, es decir sus capas y conexiones entre ellas, tras esto aparece el diseño de la base de datos, con las tablas que contendrá y cómo interactúan, y por último un mock up con bocetos iniciales de la estructura y composición de las páginas del proyecto.

El **punto número seis** desarrolla la implementación de proyecto dividiéndose en dos partes que son el backend y el frontend, donde se explicará cómo se han desarrollado diferentes funcionalidades de la página como pueden ser el login, creación de eventos, búsqueda e inscripción.

El **punto número siete** consistirá en capturas e imágenes donde se muestra como funciona la página, mostrando los resultados obtenidos, y comprobando el correcto funcionamiento del backend con el frontend.

En el **apartado número ocho** se hace una conclusión general de cómo se ha completado el proyecto con los objetivos requeridos y futuras ideas para mejorar y completar aún más la página de eventos.

Finalmente aparece **la bibliografía**, donde se citan los libros, páginas web junto a sus autores y fecha, que me han ayudado a hacer este proyecto.

Punto 2: Estado del arte

En este apartado vamos a realizar una revisión del conocimiento existente en torno al tema, tecnología o problema concreto. En el contexto de este trabajo, se presenta un análisis de cómo ha evolucionado la gestión de eventos, qué plataformas están disponibles actualmente, qué tecnologías usan en su desarrollo y qué carencias o necesidades no están resueltas. Este análisis es fundamental para justificar la necesidad del sistema propuesto y para identificar los elementos clave que orientan su diseño.

2.1 Historia de los eventos

Antes de empezar a introducimos en la historia de los eventos, es importante saber la procedencia de la palabra “evento”, la cual procede del latín “eventus” y significa “acaecimiento o cosa que sucede” [1].

Las primeras formas de encuentros organizados se remontan a Tyre, que es una ciudad fenicia del Mediterráneo, pero fue en Delfos, una ciudad de Grecia donde se empezaron a celebrar las primeras ferias para facilitar el intercambio y la comunicación entre comunidades y como mecanismo para reunir oferta y demanda, práctica que más tarde fue adoptada y difundida durante el imperio romano en toda Europa.

Durante la Revolución Industrial los eventos adquirieron un carácter más internacional. En 1851 se acogió en Londres la primera Exposición Universal en el Crystal Palace donde Inglaterra necesitó mostrar al mundo su potencial comercial y conquistar nuevos mercados, marcando así un precedente y seguido por eventos similares en París, Viena y Nueva York. A partir del siglo XX, las ferias y eventos se diversificaron por sectores como el mobiliario, la automoción, el turismo o la tecnología. En la actualidad, los eventos abarcan desde festivales culturales y congresos profesionales hasta ferias sectoriales y encuentros sociales. Su evolución ha estado marcada por un incremento en la escala, especialización, normativas y la adopción de tecnologías digitales para su organización y difusión, este recorrido histórico muestra una transformación continua.

Hasta principios de los 2000 la forma de comunicar y enterarse de los eventos se realizaba de forma analógica y local utilizando medios físicos y tradicionales como eran carteles físicos y pancartas en calles, plazas, tableros de anuncios y tiendas, también mediante la prensa escrita, que contenía secciones dedicadas a los eventos, mediante radio y televisión [2]. Esta forma de comunicación era limitada en alcance, difícil de actualizar y poco interactiva, lo que motivó al nacimiento de plataformas web. Con la aparición de internet y tras la primera página web en 1991. Un ejemplo pionero fue

Meetup.com, lanzada en 2002 y posteriormente plataformas como Eventful y Eventbrite, que extendieron el modelo hacia la promoción masiva, el ticketing y la autogestión profesional de eventos.

2.2 Plataformas de gestión de eventos en la actualidad

Existen múltiples plataformas orientadas a la creación y descubrimiento de eventos. Tras analizarlas y ver sus funcionalidades, se puede destacar las siguientes.

Una de las plataformas más conocidas es **Eventbrite**, la cual permite a usuarios y empresas publicar eventos, gestionar inscripciones, vender entradas y recibir pagos y aunque es muy funcional y potente, requiere un registro obligatorio y está enfocada al ámbito comercial [3].

Otra alternativa es **Meetup** centrada en la creación de comunidades y grupos con intereses, permite buscar actividades grupales, pero sigue dependiendo de suscripciones y una gestión centralizada [4].

También destaca **Facebook Events**, la cuál es ampliamente utilizada pero vinculada a una red social con gran alcance pero condicionado por los algoritmos y con limitaciones en cuanto a privacidad y visibilidad [5].

Si queremos algo más cercano a nuestra provincia de Granada, en el ámbito local tenemos la plataforma **Granada Social** que funciona como una plataforma múltiple donde organizaciones presentan actividades, además de noticias, podcasts y redes. Sin embargo tiene un enfoque muy específico en eventos y proyectos del ámbito social, tiene una presencia en redes muy limitada y carece de un buscador potente, sin funcionalidades para inscripción automática, valoración, ni creación libre de eventos [6].

Existen muchas otras como **Entradium**, **Tickety**, **Eventbrite Local** las cuales están enfocadas en ticketing o actividades específicas.

Estas plataformas suelen ser genéricas, comerciales o demasiado complejas para usuarios que simplemente quieren crear o encontrar eventos locales de forma rápida y directa.

2.3 Herramientas actuales en desarrollo web de eventos

En el desarrollo actual de plataformas web orientadas a la gestión de eventos, es común el uso de tecnologías modernas que garantizan rendimiento, escalabilidad y una buena experiencia de usuario. En el frontend destacan frameworks como React, Vue.js o incluso desarrollos en HTML/CSS y JavaScript puro, acompañados de librerías de diseño como Bootstrap o Tailwind CSS para construir interfaces

responsivas. En el backend, se utilizan herramientas como Node.js con Express, Spring Boot (Java) o Django (Python), que permiten construir APIs REST eficientes y estructuradas. En cuanto a bases de datos, lo más habitual es el uso de sistemas relacionales como PostgreSQL o MySQL, aunque también se emplean bases de datos no relacionales como MongoDB en proyectos con mayor flexibilidad de esquema. Para el despliegue, plataformas como Render, Vercel, Netlify o Heroku permiten publicar tanto frontend como backend en la nube de forma sencilla y accesible. Además, el uso de herramientas como Firebase (para autenticación), Stripe (para pagos), JWT (para seguridad) o GitHub (para control de versiones) forma parte habitual del ecosistema de desarrollo. El presente proyecto adopta un enfoque profesional y actualizado, integrando React para el frontend, Spring Boot para el backend, PostgreSQL como base de datos, y Render como plataforma de despliegue, lo cual lo sitúa plenamente en línea con las tecnologías más utilizadas en el sector.

Punto 3: Planificación del proyecto

Este capítulo describe la planificación seguida para llevar a cabo el proyecto. Se han definido una serie de fases que permitieron organizar el trabajo de manera estructurada, comenzando por la definición inicial de los objetivos y continuando con etapas de análisis, de desarrollo y de pruebas y correcciones hasta llegar al producto final. Esta organización ha facilitado la gestión de recursos y el control del avance hasta llegar al producto final.

Ahora se hablarán de las distintas fases que conforman el proyecto que ayudarán a tener una gestión efectiva del mismo, un buen enfoque estructurado y eficiente y por lo tanto una entrega exitosa del mismo.

3.1 Fase inicial

En esta primera etapa del proyecto se establecieron las bases fundamentales para su futuro desarrollo, para empezar y tras barajar varias ideas, se eligió el tema del proyecto junto a sus objetivos principales y, al público al que iría dirigido, además del alcance general del proyecto.

Tras un primer contacto con el tutor y validar la propuesta, se pasó a hacer una revisión del problema a resolver donde se buscaron otras plataformas similares y se hizo un análisis de los problemas y desventajas que ofrecían estas plataformas. También se investigaron las herramientas y tecnologías más comunes que empleaban esas páginas, además de otras disponibles para tomar las decisiones de cuales

usar en mi proyecto. Finalmente tras varias reuniones con el tutor y analizar las características y funciones que harían esta página mejor frente a la competencia se decidió pasar a la parte de análisis.

Esta primera fase tuvo lugar en los meses de mayo y junio con cuatro reuniones con el tutor, más unas 6-7 horas de investigación repartidas durante el mes de junio con la idea de poder empezar con las demás fases a finales de Junio y principio de Julio.

3.2 Fase de análisis

Esta fase se centra en la compensación detallada de los requisitos y necesidades del sistema, se profundizó en el estudio de necesidades funcionales y técnicas del sistema. Se analizaron las funcionalidades principales desde el punto de vista del usuario y se definieron los casos de uso, requisitos y se siguieron investigando plataformas similares como referencia.

Para empezar se estableció cuál iba a ser el funcionamiento deseado de la página, donde se distinguió entre usuarios registrados y no registrados, donde los no registrados solo podían ver los eventos, sin apuntarse o crearlos, cosa que sí iban a poder los registrados. Lo siguiente fue la elaboración de los requisitos tanto funcionales como no funcionales y los escenarios de uso, donde entran situaciones como la de crear eventos o poder apuntarse a ellos si estás registrado mediante un formulario de registro que ofrece la página, este formulario sólo te pedirá un correo, una contraseña y un nombre de usuario, para evitar tener que tratar con muchos datos personales de los usuarios y así no incumplir y no tener problemas con la ley de protección de datos, tras esto se pasó a ver que campos tendrían los eventos, donde se decidió que fueran nombre, descripción, categoría, fecha, precio, capacidad, ubicación, creador e imagen, para poder hacer búsquedas con filtros para según los gustos del usuario.

Una vez definidos los usuarios y los eventos, además de cómo iba a funcionar la página, finalmente se decidió que herramientas y arquitecturas usar, tras un proceso de estudio de las diferentes alternativas , se eligió una estructura frontend-backend, con HTML/CSS/JS + React para el frontend y Java Spring Boot para el backend, además de una base de datos PostgreSQL, para finalmente desplegarlo en un servidor Bender

Este proceso tuvo lugar durante dos semanas, con una media de cuatro-cinco horas diarias, salvo algún día.

3.3 Fase de desarrollo

Esta etapa constituyó la parte central del proyecto, en la que se implementan las funcionalidades principales del frontend y backend siguiendo los requisitos definidos previamente. Se trabajó la conexión entre ambos mediante API REST, donde se programaron los formularios de registro e inicio de sesión, la lógica de filtrado de los eventos y la gestión básica de usuarios y eventos.

Lo primero fue la construcción del backend utilizando Spring Boot, mediante un servidor embebido y una arquitectura en capas, con controladores REST para la peticiones desde el frontend, repositorios JPA para interactuar con la base de datos y modelos de datos como la clase usuario, tras esto se pasó a la creación del modelo de datos y conexión a PostgreSQL, donde se definieron entidades del sistema con anotaciones JPA y la conexión con una base de datos PostgreSQL local, se continuó con el frontend mediante HTML para la estructura del contenido, CSS para el diseño visual, JavaScript para la lógica y gestión de eventos y React para organizar la aplicación en componentes reutilizables. En este apartado se crearon las diferentes páginas de la interfaz, donde cada componente está estructurado con su propia lógica, estilos e interacción con el backend como puede ser peticiones HTTP tipo post para los formularios de registro e inicio de sesión. Finalmente y tras la estilización de la interfaz con CSS y componentes reutilizables, se desplegó en la plataforma Render para hacer accesible el backend desarrollado con Spring Boot.

Esta fase tuvo lugar durante unas tres semanas con una media de seis horas de trabajo diarias.

3.4 Fase de prueba y correcciones

En esta etapa se trató de probar y corregir lo que se ha ido desarrollando y es algo distinta ya que no se hace después de acabar la anterior sino que a lo largo del proceso de desarrollo, las pruebas se fueron realizando de manera continua durante el desarrollo proporcionando fiabilidad y rendimiento al sistema en cada momento. Cada nueva funcionalidad desarrollada fue sometida a pruebas para comprobar que cumplía con los requisitos establecidos. Se llevaron a cabo simulaciones de uso en distintos escenarios, lo que permitió detectar errores y resolverlos de forma inmediata. Entre las validaciones realizadas se incluyeron formularios de registro e inicio de sesión, comprobaciones de seguridad básica, inserción y consulta de datos en la base de datos, así como el correcto funcionamiento de los filtros y la visualización dinámica del contenido. También se corrigieron errores menores y se hicieron ajustes en la interfaz para mejorar la experiencia de usuario. Esta etapa de pruebas se extendió durante aproximadamente dos meses.

3.5 Elaboración de la memoria

Para completar el proyecto y que quede todo reflejado se procedió con la redacción de la memoria del proyecto, este ha sido un proceso largo y que se ha llevado a cabo en todo momento, para poder llevar una organización y un seguimiento del proyecto. Esta etapa no solo implica la descripción técnica del sistema sino también el análisis de los requisitos, justificación de las decisiones tomadas y la organización cronológica del desarrollo.

Se comenzó con la redacción de la introducción, motivación y objetivos, como base del proyecto y punto de partida, después se hizo la investigación y recopilación de información para el apartado de “Estado del arte”, y poder ver y comparar otras alternativas existentes y la historia de los eventos. Tras esto se fueron añadiendo las secciones de análisis, diseño, implementación según se iba avanzando además de documentación técnica de las herramientas utilizadas (frontend, backend y base de datos) con una descripción detallada de cada fase del desarrollo y su planificación.

Finalmente tuvo lugar una revisión, corrección y mejora del estilo, ortografía y coherencia del texto y la inclusión de imágenes, esquemas, tablas y fragmentos de código relevantes además de preparación de los anexos y bibliografía.

Esta fase ha permitido estructurar y presentar el trabajo de forma profesional, asegurando que tanto el funcionamiento del sistema como su proceso de desarrollo queden reflejados de forma clara y comprensible y al igual que la fase anterior ha tenido lugar durante los dos meses del proyecto con una duración de cuatro-cinco horas diarias.

Punto 4: Análisis de funcionalidad de la página

El análisis de la funcionalidad permite desglosar los diferentes aspectos del sistema para comprobar que la solución planteada responde a las necesidades reales de los usuarios. Identificar los requisitos desde el inicio resulta clave, ya que los errores en etapas tempranas son más sencillos de corregir que en fases avanzadas del desarrollo.

En esta sección se va a tratar el objetivo funcional de la plataforma, así como los usuarios que podrán participar en ella y los diferentes requisitos o necesidades del cliente, dicho de otra forma, averiguar qué es lo que el sistema debe hacer.

4.1 Objetivo funcional de la plataforma

La plataforma se orienta a la creación y descubrimiento de eventos de forma autogestionada, respondiendo a una necesidad poco atendida en el entorno digital local. Se permitirá a los usuarios registrados crear eventos propios y unirse a los ya existentes y a cualquier visitante solamente consultarlos con filtros por ubicación, fecha y categoría. Se podrán consultar los eventos entrando en ellos y así pudiendo ver información de los mismos, como el lugar, fecha exacta, precio, plazas disponibles, valoración y más. Para la creación de eventos se conducirá a los usuarios registrados a una página donde podrán rellenar una serie de campos hasta crear el evento que desean. El sistema llevará un conteo de los usuarios que se han inscrito en eventos hasta llegar a la capacidad máxima ofrecida de ese evento.

4.2 Usuarios

Este sistema no destaca por involucrar muchos tipos de actores. Dentro de nuestra página podremos diferenciar roles, sobre todo para la seguridad de la misma donde tendremos a los usuarios invitados (no registrados) que podrán ver los eventos y navegar por la página pero no podrán apuntarse a ellos, ni crear nuevos.

Por otro lado estarán los usuarios registrados mediante un correo electrónico, un nombre de usuario y una contraseña, estos usuarios tendrán la posibilidad de poder apuntarse y crear eventos, además de dejar una valoración a los creadores.

En la siguiente ilustración podemos ver el diagrama de caso de uso del Sistema y ver cómo estos usuarios interactúan con el sistema.

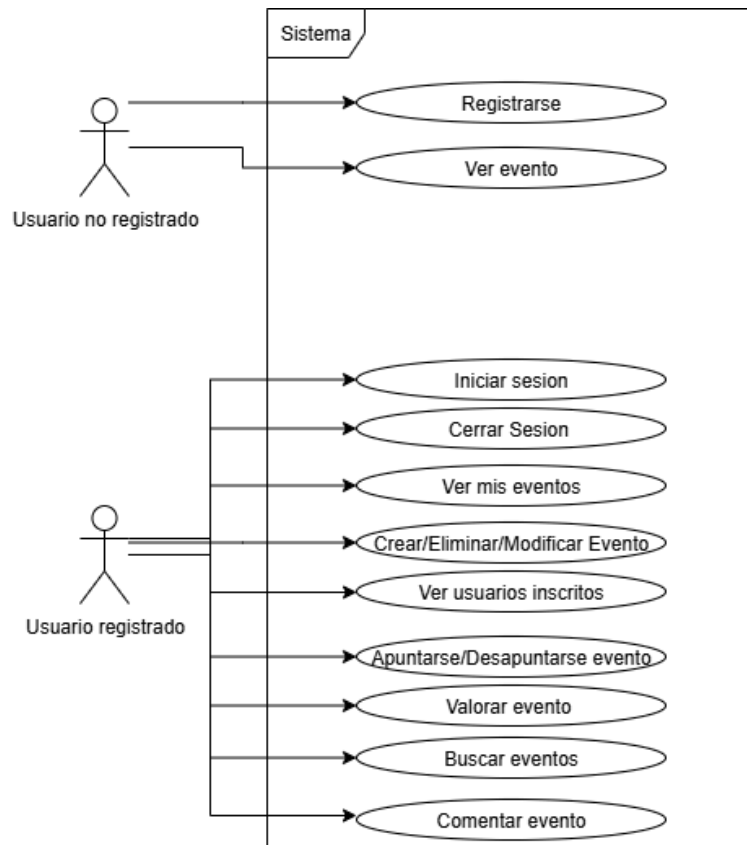


Ilustración 1: Diagrama de casos de uso del sistema

4.3 Análisis de requisitos

4.3.1 Requisitos funcionales

Subsistema de usuarios

RF1: Registrarse

Permite al usuario crear una cuenta proporcionando el correo, nombre de usuario y contraseña.

RF2: Iniciar sesión

Autentica a un usuario con sus credenciales para acceder a funciones exclusivas.

RF3: Cerrar sesión

Finaliza la sesión activa del usuario y lo redirige a la vista pública.

RF4: Cambiar contraseña

Se podrá cambiar la contraseña en la página a través de un código que llegará al email introducido.

RF5: Ver mis eventos

Muestra al usuario los eventos que ha creado o a los que se ha inscrito.

Subsistema de eventos

RF6: Crear evento

Permite a un usuario registrado publicar un evento rellenando un formulario con los datos necesarios.

RF7: Eliminar evento

El creador del evento puede eliminarlo de la plataforma.

RF8: Modificar evento

El creador del evento podrá modificar los datos del evento.

RF9: Apuntarse evento

Permite a un usuario registrado inscribirse en un evento siempre que haya plazas disponibles.

RF10: Desapuntarse de un evento

Permite a un usuario que está apuntado a un evento, quitarse de él.

RF11: Ver evento

Muestra la información completa de un evento, incluyendo ubicación, descripción, capacidad, precio y valoración, y la opción de inscribirse.

RF12: Buscador eventos

Permite a los usuarios establecer filtros para los eventos como la ciudad, categoría, precio y día del evento.

RF13: Valorar evento

El usuario podrá puntuar del 1 al 10 al evento al que ha asistido, haciendo que haya una media de valoración del evento y que cada creador tenga una media propia de la media de sus eventos creados y valorados.

RF14: Comentar evento

Un usuario que ha participado en un evento, podrá dejar un comentario visible para el resto de usuarios.

4.3.2 Requisitos no funcionales

RNF : Usabilidad

La interfaz debe ser intuitiva y fácil de usar para todo tipo de usuarios,

RNF : Rendimiento

La plataforma debe responder con rapidez incluso con múltiples usuarios simultáneamente

RNF : Seguridad

Debe proteger la información del usuario y evitar accesos no autorizados

RNF: Fiabilidad

Debe ser estable y no fallar ante acciones comunes de usuarios.

RNF: Mantenibilidad

El código debe estar estructurado para facilitar futuras modificaciones o mejoras.

RNF: Escalabilidad

La arquitectura debe permitir aumentar su capacidad su crece el número de usuarios o eventos.

RNF: Compatibilidad

La web funcionará correctamente en los navegadores y dispositivos más utilizados.

4.4 Modelos de casos de uso y escenarios de uso

Se van a examinar los escenarios de uso que involucran a los usuarios que interactúan con la plataforma que como comentábamos antes podían estar o no registrados

EU-01: Usuario no registrado consulta eventos

Un visitante entra en la página principal. Usa el buscador para filtrar eventos y explora la información de varios eventos, pero no puede inscribirse porque no está registrado.

EU-02: Usuario se registra para crear eventos

Un usuario nuevo accede al formulario de registro, introduce su correo, nombre de usuario y contraseña. Tras registrarse correctamente, el sistema lo redirige a la página principal donde ahora ve la opción de crear eventos.

EU-03: Usuario inicia sesión

Un usuario que ya tiene cuenta accede al formulario de inicio de sesión. Introduce sus credenciales. Si son correctas, el sistema lo reconoce y le da la bienvenida. Ahora puede crear eventos o apuntarse a ellos.

EU-04: Usuario crea un nuevo evento

Un usuario registrado pulsa el botón "Crear evento". Se le presenta un formulario donde indica nombre del evento, fecha, ubicación, capacidad, categoría, descripción, precio, una imagen y autor. Al enviarlo, el evento queda publicado.

EU-05: Usuario se apunta a un evento

Al navegar entre los eventos disponibles, el usuario registrado selecciona uno que le interesa. Comprueba que quedan plazas disponibles y pulsa “Apuntarse”. El sistema lo registra como inscrito y actualiza el número de plazas libres.

EU-06: Usuario valora un evento

Un usuario tras haber finalizado el evento podrá ir a la pestaña de mis eventos, y dejar una valoración al evento.

EU-07: Usuario comenta un evento

Un usuario tras haber finalizado el evento podrá ir a la pestaña de mis eventos, y dejar un comentario al sobre el evento.

EU-08: Filtro avanzados de eventos

Un usuario usa la columna de la izquierda para buscar eventos según sus gustos. El sistema actualiza los eventos mostrados automáticamente según los filtros seleccionados.

4.5 Ley de protección de datos

La protección de datos se entiende como el conjunto de principios, normativas legales y medidas destinadas para salvaguardar el derecho fundamental de las personas a controlar el uso de su información personal, esto implica que cualquier dato que identifique o haga identificable a un individuo debe recopilarse, almacenarse, gestionarse y eliminarse de manera segura y respetando su consentimiento [7].

En el ámbito europeo y español, todas las entidades que operan con información de residentes deben cumplir todos los principios y requisitos generales establecidos por el Reglamento General de Protección de Datos (RGPD) y las especificaciones y particularidades de la Ley Orgánica de Protección de datos y Garantía de los Derechos Digitales (LOPDGDD).

La plataforma desarrollada en este trabajo cumple con los principios básicos del Reglamento General de Protección de Datos (RGPD) de la Unión Europea(UE) y de la Ley Orgánica 3/2018 (LOPDGDD), el sistema de registro de usuarios requiere la recogida de datos personales como el nombre de usuario, correo electrónico y la contraseña. Estos datos son tratados con el único fin de permitir el acceso a las funcionalidades privadas de la aplicación (como la inscripción a eventos o creación de los mismos). La página garantiza el almacenamiento seguro de dicha información, implementando medidas de cifrado y restringiendo el acceso al personal no autorizado. Además se informa al usuario sobre sus derechos en materia de protección de datos, tales como el acceso, la rectificación o la eliminación de sus datos, asegurando su cumplimiento mediante políticas de privacidad y consentimiento explícito en el momento del registro [8].

4.6 Análisis de tecnologías

Para el desarrollo de este proyecto y para cubrir tanto el Backend como el Frontend, y tener una página robusta y funcional se han utilizado diferentes lenguajes y herramientas los cuales se describirán a continuación.

4.6.1 Frontend

El frontend representa la parte visual e interactiva del sitio web, es la que ve y utiliza el usuario final. En este proyecto se ha construido utilizando **HTML**(HyperText Markup Language), **CSS**(Cascading Style Sheets) y **JavaScript**. Podemos decir que es indispensable usar uno de ellos sin acabar usando los otros dos, ya que se complementan entre ellos.

HTML es el lenguaje de marcado que se utiliza para estructurar el contenido de una página web. La versión HTML5 incorporó etiquetas semánticas que permiten organizar mejor la información y facilitan tanto la accesibilidad como el posicionamiento en buscadores. Puede entenderse como el esqueleto de un sitio web, ya que define elementos como títulos, párrafos, enlaces, formularios o secciones.

Por su parte, **CSS** es el lenguaje que se encarga de dar estilo a los elementos definidos en HTML. Con CSS3 se pueden configurar aspectos como colores, tamaños, márgenes, tipografías o animaciones, además de diseñar interfaces adaptables a distintos dispositivos (responsive design). Una de sus ventajas principales es que separa la presentación visual del contenido, lo que facilita el mantenimiento del código.

En cuanto a **JavaScript**, se trata de un lenguaje de programación que se ejecuta directamente en el navegador y que aporta dinamismo e interactividad a las páginas web. Permite validar formularios,

realizar peticiones al servidor (fetch/AJAX), manipular el DOM en tiempo real o crear componentes reactivos. Al estar orientado a objetos, favorece la reutilización y modularización del código [9]

La elección de HTML5, CSS3 y JavaScript responde a su compatibilidad, versatilidad y popularidad. Son tecnologías estándar, soportadas por todos los navegadores modernos y ampliamente utilizadas en el desarrollo web actual. Además de que dispongo de un alto manejo y conocimiento de estas herramientas al haberlas utilizado en el grado.

React es una librería de JavaScript de código abierto desarrollada por Meta (antes Facebook) en 2013. Su objetivo es facilitar la construcción de interfaces de usuario dinámicas mediante el uso de componentes reutilizables, lo que resulta especialmente útil en aplicaciones de una sola página (SPA). Una de sus principales características es el uso de un DOM virtual, que permite identificar qué partes de la interfaz han cambiado y actualizar únicamente esos elementos, optimizando así el rendimiento [10]. Además de esta eficiencia, React se basa en un enfoque declarativo y cuenta con una amplia comunidad de desarrolladores, abundante documentación y herramientas maduras, lo que explica su popularidad en la industria.

En este proyecto se ha optado por React porque permite dividir la aplicación en componentes pequeños y manejables (como el formulario de registro, la cabecera, los filtros o la lista de eventos). Este enfoque modular mejora el mantenimiento del sistema y favorece su escalabilidad. A ello se suma que se trata de una tecnología muy demandada en el mercado actual, con una comunidad activa que ofrece soporte y estabilidad [11].

4.6.2 Backend

El backend se ha desarrollado en **Java** utilizando el framework **Spring Boot**, una herramienta de código abierto que forma parte del ecosistema Spring y cuyo propósito es simplificar la creación de aplicaciones web. Una de sus principales ventajas es que automatiza gran parte de la configuración necesaria, evitando al desarrollador tareas repetitivas y permitiendo centrarse en la lógica de negocio. Al incorporar configuraciones predeterminadas basadas en buenas prácticas, es posible ejecutar la aplicación sin necesidad de servidores externos adicionales.

Otro aspecto destacable es su integración con sistemas de bases de datos mediante Spring Data JPA, que proporciona una capa de abstracción para realizar operaciones CRUD sin necesidad de redactar consultas SQL de forma manual [12].

La elección de Spring Boot para este proyecto responde a su capacidad para estructurar el código en capas bien diferenciadas (modelos, controladores, repositorios), lo que mejora la organización y el mantenimiento. Además, gracias a anotaciones como `@Entity` y `@Repository` es posible mapear directamente las entidades Java en tablas de base de datos, mientras que `@RestController` y `@RequestMapping` facilitan la definición de endpoints que intercambian datos en formato JSON con el frontend [13].

4.6.3 Base de datos

Respecto a la base de datos, se ha usado PostgreSQL que es un gestor que trabaja con bases de datos relacionales y que está orientado a objetos. Es un programa de código abierto, con una gran comunidad de desarrolladores que trabajan en mejorar el programa, emplea un lenguaje SQL basado en el estándar ISO/IEC, cumple el modelo ACID(Atomicidad, Consistencia, Aislamiento y Durabilidad). Además permite definir datos que el programa no proporciona de serie y ofrece una gran escalabilidad. Se ha escogido esta base de datos debido a ventajas, como que su instalación y uso es gratis, es compatible en numerosos sistemas operativos y se puede desplegar en múltiples servidores web, tiene una configuración fácil, gran cantidad de opciones avanzadas y alta fiabilidad y robustez, se integra fácilmente con frameworks como Spring Boot, gracias a su soporte total para JDBC y JPA y gracias a la clases de Java (`@Entity`) la creación de tablas puede ser automática. PostgreSQL soporta la sintaxis estándar de SQL pero también añade funciones avanzadas como JSON, búsqueda por texto, funciones definidas por el usuario e índices personalizados [14].

4.6.4 Servidor

Tras analizar varias opciones, finalmente se va a utilizar Render como servidor donde desplegar la página. Render es un servicio que permite a los desarrolladores desplegar y alojar aplicaciones web, bases de datos y otros tipos de servicios en la infraestructura de la nube de forma sencilla y escalable.

Render ha sido una elección adecuada como servidor para desplegar este proyecto web de gestión de eventos por varias razones. En primer lugar, ofrece una integración sencilla y directa con GitHub, lo que facilita el despliegue continuo del backend desarrollado con Spring Boot y del frontend construido con React. Además, Render permite alojar de forma gratuita tanto aplicaciones web como bases de datos

PostgreSQL, lo cual es ideal para proyectos académicos que requieren funcionalidad completa sin incurrir en costes. Su interfaz es intuitiva, y la posibilidad de configurar variables de entorno facilita la gestión segura de credenciales y URLs. Otro aspecto relevante es que Render soporta conexiones HTTPS por defecto, lo que garantiza una comunicación segura entre el cliente y el servidor [15]. En definitiva, Render proporciona una solución práctica, accesible y profesional para el despliegue de aplicaciones full-stack, cumpliendo con las necesidades técnicas y pedagógicas de un Trabajo de Fin de Grado .

Punto 5: Diseño

Durante la fase de diseño se definen las soluciones técnicas que darán forma a los requisitos planteados en el análisis. Una buena planificación arquitectónica permite organizar el sistema en módulos claros, facilitando su comprensión y mantenimiento. En este capítulo se describen la arquitectura general, el diseño de la base de datos y las interfaces de usuario, que resultan clave para la interacción final con el sistema.

5.1 Modelo y arquitectura del sistema

El sistema se ha implementado siguiendo un modelo cliente-servidor, organizado en tres capas diferenciadas: aplicación, lógica de negocio y persistencia de datos. Esta división permite que la aplicación sea más sencilla de mantener, escalar y ampliar en el futuro.

La capa de **aplicación** corresponde al frontend, desarrollado con HTML, CSS, JavaScript y React. Se encarga de la interacción con el usuario, incluyendo la navegación y validaciones básicas en los formularios.

La capa de **lógica de negocio** se ha implementado con un backend en Java usando Spring Boot. Esta capa gestiona las reglas principales de la aplicación, valida la autenticación de usuarios y expone una API REST que permite crear, consultar, modificar e inscribir eventos.

Por último, la capa de **persistencia** se ocupa de la comunicación con la base de datos PostgreSQL. Se ha desarrollado con Spring Data JPA, que proporciona repositorios capaces de realizar operaciones CRUD sin necesidad de escribir consultas SQL manuales. Los repositorios actúan como puente entre los

controladores y la base de datos, devolviendo entidades que después procesan la lógica de negocio. Gracias a esta separación, el acceso a los datos se encuentra desacoplado del resto del sistema, lo que mejora la modularidad.

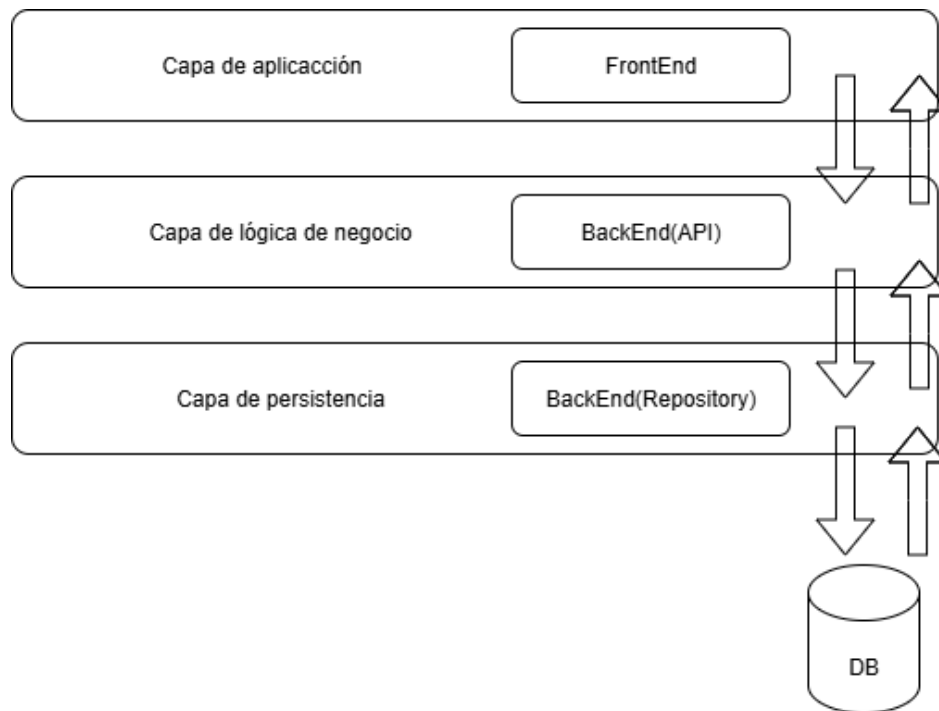


Ilustración 2: Capas del sistema

5.2 Diseño de la base de datos

En esta sección se describe el diseño de la base de datos, detallando las tablas principales junto con sus atributos y la forma en que se relacionan entre sí. El objetivo ha sido construir un modelo que garantice un almacenamiento eficiente y que facilite la recuperación de la información. Para ello, se ha seguido un proceso de normalización hasta alcanzar la tercera forma normal, lo que asegura que los datos estén organizados de manera consistente y sin redundancias innecesarias. Además, se han definido relaciones claras entre las distintas entidades, de modo que las consultas resulten más sencillas y fiables. Este planteamiento proporciona una base sólida, flexible y preparada para futuras ampliaciones del sistema.

Se ha utilizado PostgreSQL como sistema gestor de bases de datos por su robustez, fiabilidad y compatibilidad con Spring Boot y por seguir buenas prácticas con JPA/Hibernate, ya que el uso de

anotaciones JPA(@Entity, @Id, @GeneratedValue, etc.) permite mapear directamente las clases Java con las tablas de bases de datos, facilitando enormemente el desarrollo y el mantenimiento del proyecto.

La tabla **Usuario** representa a los usuarios que interactúan con el sistema, ya sea para registrarse, iniciar sesión, crear eventos o inscribirse en los ya existentes. Esta tabla es esencial para gestionar la autenticación y la personalización del contenido. Cada registro en esta tabla corresponde a un usuario único y se compone de los siguientes campos principales:

id: Identificador único generado automáticamente para cada usuario. Actúa como clave primaria.

correo: Dirección de correo electrónico del usuario. Es un dato único que permite identificar y contactar al usuario, y es necesario para realizar el proceso de autenticación.

nombreUsuario: Nombre visible que se utiliza para mostrar al usuario dentro del sistema (por ejemplo, en los saludos o en los eventos creados).

contraseña: Cadena alfanumérica utilizada para proteger el acceso a la cuenta del usuario. Esta información se guarda en la base de datos y debe gestionarse con medidas de seguridad (en producción, debería almacenarse de forma cifrada).

valoraciónMedia: Valor numérico que presenta la media de las valoraciones obtenidas por los eventos que ha creado el usuario.

numeroValoraciones: número total de veces que han valorado algún evento de ese usuario.

La tabla Usuario es clave en la relación con otras tablas, ya que puede estar vinculada con múltiples eventos como creador. Puede inscribirse en varios eventos, lo que establece una relación muchos a muchos con la tabla Evento a través de la tabla intermedia Inscripción.

La tabla **Evento** contiene toda la información relativa a los eventos disponibles en la plataforma. Cada fila representa un evento creado por un usuario y que puede ser consultado o al que se pueden inscribir otros usuarios. Los campos que componen esta tabla son:

id: Identificador único del evento, generado automáticamente. Es la clave primaria.

nombre: Título del evento, utilizado como encabezado en la vista previa y en los detalles del evento.

descripción: Texto explicativo donde se detalla en qué consiste el evento, el lugar, los requisitos, el objetivo, etc.

categoría: Clasificación temática del evento (por ejemplo: Música, Deporte, Gastronomía, etc.). Este campo permite facilitar la búsqueda y organización de los eventos.

fecha: Fecha concreta en la que se va a celebrar el evento. Es esencial para ordenar y filtrar cronológicamente.

precio: Coste de inscripción o asistencia al evento, expresado en euros. Puede ser 0 para eventos gratuitos.

capacidad: Número máximo de participantes permitidos. Este valor se actualiza cuando los usuarios se inscriben.

ubicación: Dirección o ciudad donde se celebrará el evento.

imagen: URL de una imagen representativa del evento, utilizada para mejorar la presentación visual en la interfaz.

creador: Hace referencia al nombre del usuario que ha creado el evento, lo que establece una relación con la tabla Usuario.

valoraciónMedia: Nota media del evento, calculada a partir de las valoraciones de los asistentes.

numeroValoraciones: Número de valoraciones recibidas por el evento, necesario para calcular la media.

La tabla Evento se relaciona con otras entidades, tendrá una relación uno a muchos con Usuario, ya que un usuario puede crear múltiples eventos, pero un evento es creado por un único usuario. Relación muchos a muchos con Usuario a través de la tabla Inscripción, que indica qué usuarios se han apuntado a qué eventos.

La tabla **Inscripción** actúa como una entidad intermedia que permite modelar la relación muchos a muchos entre Usuario y Evento. Es decir, un usuario puede inscribirse en muchos eventos y un evento tendrá muchos usuarios inscritos. Los campos que contiene son:

usuario_id: Clave foránea que referencia al usuario inscrito

evento_id: Clave foránea que referencia al evento inscrito

fecha: Fecha que se apuntó al evento

valoracion: Valoración del usuario al evento que ha participado y ha finalizado, siendo única.

comentario: Comentario dejado por un usuario que ha asistido al evento, siendo único.

Este modelo permite que los usuarios registrados puedan valorar los eventos una única vez, cuando hayan finalizado, y que estas valoraciones afecten a la media del evento como a la media del creador del evento. Se ha mantenido una estructura normalizada y eficiente que permite extender el sistema fácilmente.

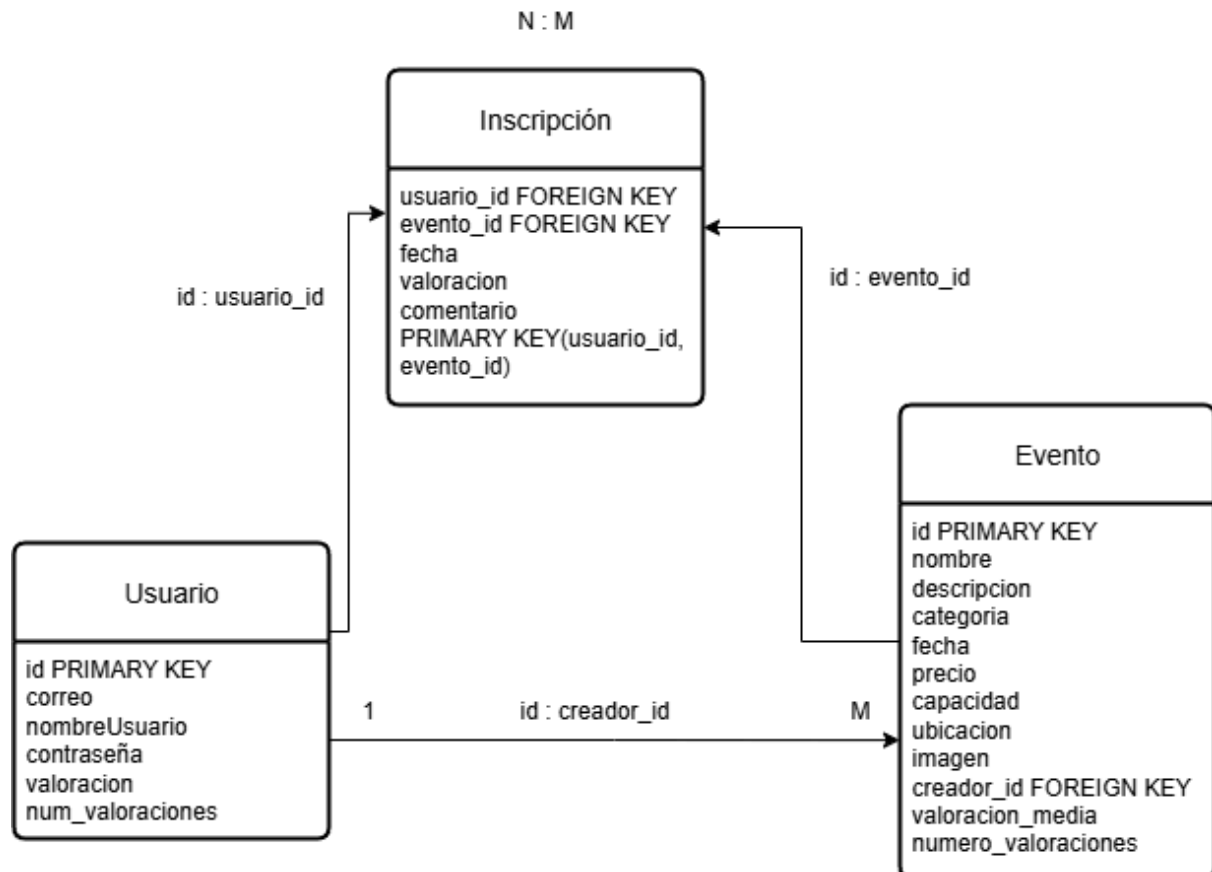


Ilustración 3: Tablas del sistema

5.3 Diseño de las interfaces

Se van a incluir y comentar los mockups de las diferentes páginas que tendrá el sistema. Esto será una guía de aquello que se requiere que tenga la página, incluyendo las funcionalidades mínimas que desea ver en cada página

En primer lugar nuestra página ofrecerá arriba a la derecha la posibilidad de iniciar sesión ó registrarse que nos llevarán a una página con los formularios correspondientes, a la izquierda aparecerá el título y un buscador por ciudad, debajo una frase con una foto de presentación. La estructura principal de la página consta de dos partes, a la izquierda tendrá una columna con las opciones de búsqueda y a la derecha aparecerán los eventos disponibles en el momento, como podemos ver a continuación.



Ilustración 4: Mockup Página Principal

Una vez iniciada la sesión, nos aparecerá un mensaje de bienvenida y la opción de crear evento arriba a la derecha, como podemos ver a continuación.



Ilustración 5: Mockup Página Principal con Sesión Iniciada

Al pinchar en crear evento nos llevará a un formulario donde podremos rellenar los distintos datos de nuestro evento que tras darle a crear aparecerá en la página principal, aquí aparecerán solamente con la foto, el título, la categoría y la fecha y un botón de más información que nos llevara a otra página donde a

la izquierda podremos ver la foto y a la derecha toda la información con finalmente la opción abajo de inscribirse al evento si has iniciado sesión.



Ilustración 6: Mockup Evento en Página Principal

Al darle a más información nos llevará a:



Ilustración 7: Mockup Detalles evento

Al pinchar en el mensaje de Bienvenido usuario, se desplegará un menú con dos opciones la primera será la de ver mis eventos que nos llevará a otra página donde se dividirá en dos columnas, a la izquierda aparecerán los eventos a los que te has inscrito con la opción de desapuntarse o de valorar al

creador si ya ha pasado el evento, y a la derecha los eventos creados por ti con la opción en todo momento de modificarlos o eliminarlos, la otra opción del menú será la de cerrar sesión.

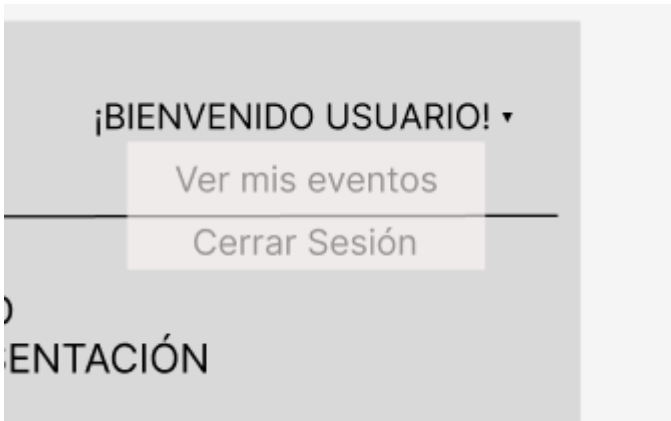


Ilustración 8: Mockup Menú Usuarios

Si pulsamos en ver mis eventos se verá algo así donde tendremos los eventos inscritos con la opción de desapuntarse si no ha ocurrido aún y la posibilidad de poder dejar una valoración y un comentario y de ver la media de valoraciones y los comentarios, una vez finalizado. A la derecha podremos modificar eliminar y ver los usuarios inscritos en los eventos creados.

EVENTOS ESPAÑA			BUSCADOR DE CIUDAD			CREAR EVENTO			Inicio			¡BIENVENIDO USUARIO! ▾		
Eventos inscrito						Eventos creados								
Evento de ... info evento		Evento de ... info evento		Evento de ... info evento		Evento de ... info evento		Evento de ... info evento		Evento de ... info evento				
Desapuntarse		Desapuntarse		Desapuntarse		Modificar Eliminar		Modificar Eliminar		Modificar Eliminar				

Ilustración 9: Mockup Página Mis Eventos

Punto 6: Implementación

Con el análisis y el diseño ya definidos, en este capítulo se aborda la fase de implementación del sistema, en la que se ha traducido la planificación previa en código funcional. Dado que el proyecto es extenso, no se presenta el desarrollo completo línea por línea, sino ejemplos representativos que permiten comprender cómo se resolvieron los problemas técnicos más relevantes.

La implementación se ha dividido en dos bloques principales: el backend, desarrollado en Java con Spring Boot, y el frontend, construido con React. Ambos se comunican mediante peticiones HTTP siguiendo una arquitectura REST, lo que permite a la interfaz interactuar con la base de datos de manera estructurada.

Para el desarrollo se ha utilizado Visual Studio Code como entorno de programación, una herramienta con la que ya contaba con experiencia previa gracias a distintas asignaturas del grado y que ha facilitado la construcción y depuración del código.

6.1 Backend

El backend del proyecto como se ha comentado antes está desarrollado con Spring Boot, un framework ampliamente reconocido en la comunidad de desarrollo Java por su capacidad para simplificar la creación de aplicaciones web robustas y escalables. organizado siguiendo una estructura por capas, dividiendo responsabilidades en diferentes apartados.

El backend estará dividido en cuatro partes, como son **model**, **repository**, **controller** y **service**. Esta separación sigue el principio de responsabilidad única y facilita el mantenimiento y escalabilidad del sistema. Tendrá una estructura de la siguiente forma:

```
src/  
└─ main/  
    └─ java/  
        └─ com/miweb/eventos/  
            ├── controller/  
            ├── model/  
            └── repository/
```



Ilustración 10: Estructura Backend

- **Model** (model/): En este apartado encontraremos las clases que representan las entidades de la base de datos, donde cada clase define sus atributos, claves primarias y relaciones. Las entidades principales son **Usuario.java** que representara a los usuarios registrados en la plataforma, **Evento.java** que define los eventos creados por los usuarios e **Inscripcion.java** que modela la relación entre usuarios y eventos a los que se inscriben. Gracias a esta estructura se asegura que un usuario solo pueda inscribirse y valorar una única vez cada evento.

Cada clase está anotada con `@Entity` y se han utilizado anotaciones como `@Id` y `@GeneratedValue` para definir las claves primarias y la generación automática de los IDs.

Aquí podemos ver como se ha implementado la tabla de usuario con sus datos y debajo irán todos los getters y setters:

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String correo;
    private String nombreUsuario;
    private String contraseña;
    private double valoracionMedia;
    private int numeroValoraciones;

    // Getters y Setters
```

Código 1: Clase Usuario

- **Repository** (repository/): En esta carpeta tendremos las interfaces que extienden de JpaRepository, lo que permite realizar operaciones CRUD automáticamente. Estas interfaces interactúan directamente con la base de datos y tendremos **UsuarioRepository.java** que permite encontrar usuarios por ID o por nombre de usuario o por correo y contraseña, **EventoRepository.java** que permite buscar eventos por ID, por nombre o por creador y por último **InscripcionRepository.java** que gestiona las inscripciones, permitiendo buscar por usuario, por evento y verificar si una inscripción ya existe.

A continuación podemos ver los métodos para UsuarioRepository.java:

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {  
    boolean existsByCorreo(String correo);  
  
    Optional<Usuario> findByCorreoAndContraseña(String correo, String contraseña);  
    Optional<Usuario> findByNombreUsuario(String nombreUsuario);  
}
```

Código 2: Métodos Repositorio de Usuario

- **Controller** (controller/): La capa controller se encarga de exponer los endpoints de la API REST que permite al frontend interactuar con el sistema. Se han definido tres controladores principales como son **UsuarioController.java** que gestiona el registro e inicio de sesión. Comprueba las credenciales y devuelve el objeto al usuario para almacenarlo en el frontend. Tenemos también **EventoController.java** el cual tiene bastantes funcionalidades y permite crear, listar, modificar y eliminar eventos. Además incluye la funcionalidad de buscar eventos según distintos filtros como la ciudad, categoría, precio y fecha. Por último tenemos **InscripcionController.java** que gestiona la inscripción de usuarios a eventos, asegura que no se puede uno inscribir dos veces y que no haya más inscripciones que plazas disponibles. Además gestiona la funcionalidad de valorar eventos, actualizar medias y evitar valoraciones duplicadas.

Aquí tenemos las funcionalidades de login y registro de usuario y debajo la de crear y listar de eventos:


```

@PostMapping("/registro")
public String registrar(@RequestBody Usuario usuario) {
    if (repo.existsByCorreo(usuario.getCorreo())) {
        return "Correo ya registrado";
    }
    repo.save(usuario);
    return "Usuario registrado correctamente";
}

@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody Map<String, String> datos) {
    String correo = datos.get(key:"correo");
    String contraseña = datos.get(key:"contraseña");

    Optional<Usuario> usuario = repo.findByCorreoAndContraseña(correo, contraseña);

    if (usuario.isPresent()) {
        Map<String, Object> response = new HashMap<>();
        response.put(key:"id", usuario.get().getId());
        response.put(key:"nombreUsuario", usuario.get().getNombreUsuario());
        return ResponseEntity.ok(response);
    } else {
        return ResponseEntity.status(status:401).body(body:"Credenciales incorrectas");
    }
}

```

Código 3: Funcionalidades de Registro y Login de Usuarios

```

@PostMapping("/crear")
public String crearEvento(@RequestBody Evento evento) {
    eventoRepo.save(evento);
    return "Evento creado correctamente";
}

@GetMapping("/listar")
public List<Evento> obtenerEventos() {
    return eventoRepo.findAll();
}

```

Código 4: Funcionalidades de Crear y Listar eventos.

- **Service** (/service): En este apartado tendremos el archivo EmailService.java que su principal funcionalidad será la de enviar correos electrónicos con códigos de recuperación de contraseña a los usuarios que lo soliciten. Para ello usa la clase JavaMailSender de Spring para enviar un correo sencillo al destinatario proporcionado. Se envía con un asunto fijo ("Código de recuperación") y un cuerpo personalizado que incluye el código generado. Este correo se crea desde la cuenta creada eventostfg@gmail.com mediante los parámetros SMTP definidos en application.properties

- Tenemos un último apartado en el backend que es la conexión a la base de datos, que en este caso es PostgreSQL y está conectada mediante los datos del archivo application.properties, con esta configuración, Spring Boot se encarga de generar automáticamente las tablas a partir de las entidades si estas no existen, permitiendo un desarrollo rápido y sincronizado con el modelo de datos, además configurar los parámetros para enviar los mails de recuperación de contraseña, podemos verlo a continuación:

```
1  spring.application.name=eventos
2
3  # PostgreSQL
4  spring.datasource.url=jdbc:postgresql://localhost:5432/eventos
5  spring.datasource.username=postgres
6  spring.datasource.password=franquiles
7
8  # JPA
9  spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
12
```

Código 5: Conexión a la base de datos

6.2 Frontend

El frontend de la aplicación ha sido implementado utilizando React.js, una biblioteca de JavaScript desarrollada para construir interfaces de usuario de forma eficiente y modular. Esta elección se basa en su gran comunidad, facilidad para manejar estados dinámicos y su compatibilidad , lo cual permite una experiencia fluida sin recargar la página constantemente.

Cada vista o funcionalidad principal se encuentra dentro de la carpeta pages/ y cada componente compartido(como encabezado o pie de página) dentro de components/. El archivo App.js se encarga de definir todas las rutas de navegación mediante react-router-dom.

frontend/

```
|— public/
|   |— imagenes/
|— src/
|   |— components/ → Navbar.js, Footer.js
|   |— pages/      → HomePage.js, CrearEvento.js, MisEventos.js, etc.
|   |— assets/
|   |— App.js
```

Ilustración 11: Estructura Frontend

La conexión con el backend se realiza mediante peticiones HTTP, utilizando `fetch()` para realizar operaciones GET, POST, PUT, DELETE hacia los endpoints de la API REST creada en Spring Boot. Todas las rutas apuntan a <http://localhost:8080/api/>

Ahora vamos a repasar las principales páginas que se han implementado con sus funcionalidades y requisitos que cumplen.

Empezamos con `Navbar.js` la cual será el header de la página y contendrá los enlaces a los formularios de inicio de sesión y registro, además de un buscador por ciudad permitiendo que solo salgan eventos en esa ciudad y poder luego aplicar otros filtros ya solo sobre esa ciudad y la opción de crear eventos. Una vez que se ha iniciado sesión.

6.2.1 Registro

Al darle a registrarse te llevará a **RegisterPage.js** que tendrá un formulario donde recogerá los datos como correo, nombre de usuario y contraseña y los envía al backend en formato JSON mediante `fetch()`, en una petición POST al endpoint, si el registro fue exitoso se guarda el nombre de usuario en `localStorage` y se redirige a la página principal.

```

function RegisterPage() {
  const [formData, setFormData] = useState({
    correo: "",
    nombreUsuario: "",
    contraseña: ""
  });

  const navigate = useNavigate();

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch("http://localhost:8080/api/usuarios/registro", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData)
    });
    const msg = await res.text();

    if (msg === "Usuario registrado correctamente") {
      localStorage.setItem("usuario", formData.nombreUsuario);
      navigate("/");
    } else {
      alert(msg);
    }
  };
}

```

Código 6: Registro de Usuarios en el Frontend

6.2.2 Login

Si pulsamos en iniciar sesión nos llevará a LoginPage.js, donde podremos meter nuestro correo y contraseña, que al introducirlo y enviar el formulario se ejecuta una función que realiza una petición HTTP de tipo POST al endpoint del backend, enviando los datos en formato JSON, si las credenciales son correctas, el backend responde con los datos del usuario, los cuales se almacenan en localStorage para mantener la sesión activa.

```

function LoginPage() {
  const [formData, setFormData] = useState({ correo: "", contraseña: "" });
  const navigate = useNavigate();

  const handleChange = (e) =>
    setFormData({ ...formData, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();

    const res = await fetch("http://localhost:8080/api/usuarios/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData)
    });

    if (res.ok) {
      const data = await res.json();
      localStorage.setItem("usuario", data.nombreUsuario);
      localStorage.setItem("usuarioId", data.id);
      navigate("/");
    } else {
      const errorMsg = await res.text();
      alert(errorMsg);
    }
  };
}

```

Código 7: Login de Usuarios en el Frontend

6.2.3 Recuperar contraseña

El proceso de recuperación de contraseña implementado en la aplicación web permite a los usuarios restablecer su acceso de forma segura y autónoma en caso de olvidar su clave. Esta funcionalidad está integrada directamente en la misma página de inicio de sesión (LoginPage.js) mediante un sistema de vistas controladas por el estado de React. Al hacer clic en “¿Olvidaste tu contraseña?”, se solicita al usuario que introduzca su correo electrónico. Una vez enviado, el backend genera un código aleatorio de 6 cifras y lo envía a dicho correo mediante el servicio EmailService mencionado anteriormente. Tras recibirlo, el usuario lo introduce en la aplicación, donde se valida contra el código previamente almacenado. Si es correcto, se le permite establecer una nueva contraseña, que debe introducir dos veces para confirmar. Finalmente, si ambas coinciden, la contraseña se actualiza en la base de datos y el usuario es redirigido al formulario de inicio de sesión. Este flujo asegura tanto la seguridad como la usabilidad del proceso, sin necesidad de intervención administrativa externa.

```

//ENVIAR CODIGO
const enviarCodigo = async () => {
  try {
    const res = await fetch(`${BACKEND_URL}/api/usuarios/recuperar`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ correo: emailRecuperar }),
    });
    const msg = await res.text();
    if (res.ok) {
      setVista("codigo");
      alert("Código enviado a tu correo");
    } else {
      alert(msg);
    }
  } catch (err) {
    alert("Error enviando el código");
    console.error(err);
  }
};

//VALIDAR CODIGO
const validarCodigo = async () => {
  try {
    const res = await fetch(`${BACKEND_URL}/api/usuarios/validar-codigo`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ correo: emailRecuperar, codigo: codigoIngresado }),
    });
    const msg = await res.text();
    if (res.ok) {
      setVista("nueva");
    } else {
      alert(msg);
    }
  } catch (err) {
    alert("Error validando el código");
  }
};

```

Código 8: Envío y Validación de código para recuperar contraseña.

```

//CAMBIAR CONTRASEÑA
const cambiarPassword = async () => {
  if (nueva1 !== nueva2) {
    alert("Las contraseñas no coinciden");
    return;
  }

  try {
    const res = await fetch(`${BACKEND_URL}/api/usuarios/cambiar-password`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ correo: emailRecuperar, nueva: nueva1 }),
    });
    const msg = await res.text();
    if (res.ok) {
      alert("Contraseña cambiada correctamente");
      setVista("login");
    } else {
      alert(msg);
    }
  } catch (err) {
    alert("Error cambiando contraseña");
  }
};

```

Código 9: Cambio de contraseña

6.2.4 Crear evento

Una vez registrado tendremos la opción de crear evento, que nos llevará a la página CrearEvento.js la cual tendrá un formulario con distintos campos que representan los atributos del evento, cada cambio en el formulario actualiza el estado local formData mediante el hook useState. Cuando el usuario menciona el formulario, se ejecuta una función que añade al objeto del evento el campo creador, con el nombre de usuario del creador y se envía al backend con una petición POST al endpoint. Si la operación es exitosa se mostrar un mensaje de evento creado correctamente.

```

useEffect(() => {
  const user = localStorage.getItem("usuario");
  if (user) {
    setUsuario(user);
  } else {
    alert("Debes iniciar sesión para crear eventos");
    navigate("/login");
  }
}, [navigate]);

const handleChange = (e) => {
  const { name, value } = e.target;

  const newValue =
    name === "precio" ? parseFloat(value) :
    name === "capacidad" ? parseInt(value) :
    value;

  setFormData({ ...formData, [name]: newValue });
};

const handleSubmit = async (e) => {
  e.preventDefault();
  const eventoConCreador = { ...formData, creador: usuario };

  const res = await fetch("http://localhost:8080/api/eventos/crear", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(eventoConCreador)
  });

  if (res.ok) {
    alert("Evento creado correctamente");
    navigate("/");
  } else {
    alert("Error al crear el evento");
  }
};

```

Código 10: Creación de Eventos en el Frontend

6.2.5 HomePage (Búsquedas y eventos)

El componente HomePage.js cumple la función de página principal de la plataforma y se encarga de mostrar los eventos disponibles, así como de gestionar el sistema de búsqueda mediante filtros dinámicos. Al cargar la página, se comprueba si hay una ciudad seleccionada previamente por el usuario (almacenada en localStorage desde el buscador del encabezado); si es así, se realiza una búsqueda filtrada en esa ciudad, y si no, se listan todos los eventos disponibles mediante una petición al backend (/api/eventos/listar). La parte izquierda de la interfaz contiene una columna de filtros donde el usuario puede seleccionar criterios como la categoría, el rango de precios, la fecha del evento y, de forma

implícita, la ciudad previamente elegida. Estos filtros actualizan el estado local del componente (useState), y al pulsar el botón “Buscar” se construye una URL con los parámetros seleccionados, que se envía al endpoint /api/eventos/busqueda del backend, devolviendo solo los eventos que coinciden. Además, se ofrece un botón de “Restablecer filtros” que limpia los criterios aplicados, manteniendo si corresponde la ciudad seleccionada. En la parte derecha se visualizan los eventos: se filtran para mostrar únicamente los eventos con fecha igual o posterior al día actual, y cada uno se representa con una tarjeta que incluye imagen, nombre, categoría, fecha y un botón para ver más información.

Además los eventos tendrán la opción de darle a más información y te llevará a otra página EventoDetalle.js donde saldrá la información completa del evento como el creador con su valoración y eventos creados pasados y la opción de inscribirse si has iniciado sesión.

La interfaz muestra un botón para inscribirse únicamente si el usuario no está inscrito aún y si quedan plazas disponibles. Al hacer clic en este botón, se lanza una ventana de confirmación y, si el usuario acepta, se envía una petición POST al endpoint /api/inscripciones/apuntarse, que incluye en el cuerpo los identificadores del usuario y del evento. Si la inscripción se realiza correctamente, se muestra un mensaje de éxito, se actualiza el estado yaInscrito y se recargan los datos del evento para reflejar la nueva capacidad disponible. Esta lógica asegura que un usuario no pueda apuntarse dos veces al mismo evento ni hacerlo si ya no quedan plazas. Todo el proceso está gestionado de forma reactiva mediante hooks (useEffect, useState) y peticiones HTTP que sincronizan constantemente el estado del cliente con los datos del servidor.

```

function EventoDetalle() {
  useEffect(() => {
    fetch(`http://localhost:8080/api/eventos/${id}`)
      .then((res) => res.json())
      .then((data) => {
        setEvento(data);

        fetch(`http://localhost:8080/api/usuarios/por-nombre/${data.creador}`)
          .then((res) => res.json())
          .then((creadorData) => setCreador(creadorData))
          .catch((err) => console.error("Error al cargar datos del creador:", err));
      })
      .catch((err) => console.error("Error al cargar evento:", err));

    if (usuarioId) {
      fetch(`http://localhost:8080/api/inscripciones/comprobar?usuarioId=${usuarioId}&eventoId=${id}`)
        .then(res => res.json())
        .then(setYaInscrito);
    }
  }, [id, usuarioId]);

  if (!evento) return <p>Cargando evento...</p>;
  const inscribirse = async () => {
    const confirmar = window.confirm("¿Quieres inscribirte a este evento?");
    if (!confirmar) return;
    const res = await fetch("http://localhost:8080/api/inscripciones/apuntarse", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        usuarioId: parseInt(usuarioId),
        eventoId: parseInt(id)
      })
    });

    const msg = await res.text();
    alert(msg);

    if (msg === "Inscripción realizada correctamente.") {
      setYaInscrito(true);
      // Recargar datos actualizados del evento (para ver capacidad)
      fetch(`http://localhost:8080/api/eventos/${id}`)
        .then((res) => res.json())
        .then((data) => setEvento(data));
    }
  };
}

```

Código 11: Detalle de los Eventos en el Frontend

6.2.6 Mis eventos

Una vez hayamos iniciado sesión nos aparecerá Bienvenido y nuestro nombre de usuario, y al pinchar ahí aparecerá un menú con dos opciones, siendo una la de cerrar sesión y la otra será mis eventos que nos llevará a una página MisEventos.js, esta estará dividida en dos columnas.

En la columna izquierda, se cargan los eventos en los que el usuario está actualmente inscrito, mediante una petición al backend que utiliza el `usuarioId` almacenado en `localStorage`. Para cada evento, si ya ha finalizado (comparando la fecha con la actual), se ofrece al usuario la posibilidad de valorar el evento a través de un botón que abre un prompt donde puede introducir una puntuación del 1 al 10. Esta puntuación se envía al backend mediante una petición PUT al endpoint `/api/inscripciones/valorar`, junto con el ID del usuario y del evento. Si ya ha sido valorado, se muestra un mensaje de agradecimiento y la

valoración media actual del evento. Además el usuario podrá dejar un comentario sobre cada evento una vez este haya finalizado. Para ello se implementa el botón comentar que abre un área de texto. Una vez escrito y enviado, el comentario se guarda mediante una petición PUT al backend `/api/inscripciones/comentar`. Para evitar comentarios duplicados, el sistema consulta desde el backend si el usuario ya ha comentado ese evento y si es así muestra el mensaje “Comentario enviado” en lugar del botón. Existe también un sistema de visualización de comentarios públicos bajo cada evento valorado, el usuario puede desplegar los comentarios existentes, los cuales se obtienen con una petición al endpoint `/api/inscripciones/comentarios/{eventoId}`. En cambio, si el evento aún no ha sucedido, el usuario puede elegir desapuntarse, lo que se realiza mediante una petición DELETE al endpoint `/api/inscripciones/desapuntarse`, actualizando inmediatamente la interfaz para eliminar ese evento de la lista. Aquí podemos ver las opciones de desapuntarse y valorar evento si ha finalizado:

```
const handleDesapuntarse = async (eventoId) => {
  const confirmar = window.confirm("¿Seguro que quieres desapuntarte de este evento?");
  if (!confirmar) return;

  const res = await fetch(`http://localhost:8080/api/inscripciones/desapuntarse?usuarioId=${usuarioId}&eventoId=${eventoId}`, {
    method: "DELETE"
  });

  const msg = await res.text();
  alert(msg);

  if (res.ok) {
    // Quitar el evento de la lista visual
    setEventosInscritos(prev => prev.filter(evento => evento.id !== eventoId));
  }
};

const handleValorar = async (eventoId) => {
  const nota = prompt("Valora el evento del 1 al 10:");
  const valor = parseInt(nota);
  if (!valor || valor < 1 || valor > 10) {
    alert("Introduce un número válido del 1 al 10");
    return;
  }

  const res = await fetch(`http://localhost:8080/api/inscripciones/valorar`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      usuarioId,
      eventoId,
      valoracion: valor
    })
  });

  const msg = await res.text();
  alert(msg);

  if (res.ok) {
    setValoraciones(prev => ({ ...prev, [eventoId]: true }));
  }
};
```

Código 12: Desapuntarse y Valorar Eventos en el Frontend

A continuación se muestra la parte del código para comentar un evento una única vez:

```

const handleComentar = async (eventoId) => {
  if (!nuevoComentario.trim()) {
    alert("El comentario no puede estar vacío.");
    return;
  }
  const res = await fetch("http://localhost:8080/api/inscripciones/comentar", {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      usuarioId,
      eventoId,
      comentario: nuevoComentario
    })
  });
  const msg = await res.text();
  alert(msg);
  if (res.ok) {
    setComentarioAbierto(null);
    setNuevoComentario("");
    setComentariosEnviados(prev => ({ ...prev, [eventoId]: true }));
    // Refrescar comentarios visibles
    if (comentariosAbiertos[eventoId]) {
      await toggleComentarios(eventoId); // cerrar
      await toggleComentarios(eventoId); // reabrir y refrescar
    }
  }
};

const toggleComentarios = async (eventoId) => {
  const abierto = comentariosAbiertos[eventoId];

  if (abierto) {
    setComentariosAbiertos(prev => ({ ...prev, [eventoId]: false }));
  } else {
    const res = await fetch(`http://localhost:8080/api/inscripciones/comentarios/${eventoId}`);
    const data = await res.json();

    setComentariosPorEvento(prev => ({ ...prev, [eventoId]: data }));
    setComentariosAbiertos(prev => ({ ...prev, [eventoId]: true }));
    setNumComentarios(prev => ({ ...prev, [eventoId]: data.length }));
  }
};

```

Código 13: Comentar Eventos en el Frontend

En la columna derecha, se muestran los eventos creados por el usuario. Estos se cargan automáticamente al montar el componente, usando su nombre de usuario para consultar al backend (/api/eventos/creados). Para cada evento, el usuario dispone de tres acciones: modificar, eliminar y ver inscritos. La opción de modificar redirige a otra ruta (/modificar-evento/{id}) donde podrá editar el evento. El botón de eliminar muestra primero una ventana de confirmación, y en caso afirmativo realiza una petición DELETE al backend, eliminando el evento de forma permanente y actualizando el listado local. Por último, el botón "Ver inscritos" permite desplegar una lista de usuarios que se han apuntado al evento. Al hacer clic, se realiza una petición al backend (/api/inscripciones/evento/{eventoId}/usuarios) que devuelve los nombres de usuario de los inscritos, los cuales se muestran debajo del evento. Si se vuelve a hacer clic sobre el mismo botón, se oculta la lista.

```

function MisEventos() {
  const [eventosCreados, setEventosCreados] = useState([]);
  const [eventosInscritos, setEventosInscritos] = useState([]);
  const [valoraciones, setValoraciones] = useState({});
  const usuarioId = localStorage.getItem("usuarioId");
  const [inscritosPorEvento, setInscritosPorEvento] = useState({});
  const [eventoAbierto, setEventoAbierto] = useState(null);
  const hoy = new Date().toISOString().split("T")[0];

  useEffect(() => {
    if (usuarioId) {
      fetch(`http://localhost:8080/api/inscripciones/inscritos/${usuarioId}`)
        .then(res => res.json())
        .then(data => setEventosInscritos(data))
        .catch(err => console.error("Error cargando eventos inscritos:", err));
    }
  }, [usuarioId]);

  useEffect(() => {
    const usuario = localStorage.getItem("usuario");
    if (usuario) {
      fetch(`http://localhost:8080/api/eventos/creados?usuario=${usuario}`)
        .then(res => res.json())
        .then(data => setEventosCreados(data))
        .catch(err => console.error("Error cargando eventos creados:", err));
    }
  }, []);
  const navigate = useNavigate();

```

Código 14: Ver Eventos Creados e Inscritos en el Frontend

```

const handleEliminarEvento = async (id) => {
  const confirmar = window.confirm("¿Estás seguro de que quieres eliminar este evento?");
  if (confirmar) {
    const res = await fetch(`http://localhost:8080/api/eventos/${id}`, {
      method: "DELETE",
    });
    if (res.ok) {
      alert("Evento eliminado correctamente");
      setEventosCreados(prev => prev.filter(evento => evento.id !== id));
    } else {
      alert("Error al eliminar el evento");
    }
  }
};

const verInscritos = async (eventoId) => {
  if (eventoAbierto === eventoId) {
    setEventoAbierto(null);
    return;
  }

  const res = await fetch(`http://localhost:8080/api/inscripciones/evento/${eventoId}/usuarios`);
  const data = await res.json();
  setInscritosPorEvento(prev => ({ ...prev, [eventoId]: data }));
  setEventoAbierto(eventoId);
};

```

Código 15: Eliminar Evento y Ver Inscritos en el Frontend

6.3 Servidor

Una vez todo diseñado y funcionando correctamente en mi entorno local, lo siguiente es desplegarlo en un servidor y que esté disponible de cara al público, en este caso Render como se comentó en apartados anteriores.

El primer paso fue subir toda mi carpeta de trabajo a GitHub, ya que con su interfaz es posible tener acceso rápido a los repositorios, haciendo más fácil el manejo y la integración de las modificaciones que se realizan. Tras esto, lo siguiente es crear las tres partes fundamentales del proyecto en Render como son el backend, frontend y base de datos.

En primer lugar se comenzó con el backend, para ello en Render se creó un nuevo “Web Service”, donde se seleccionó la carpeta que contenía el backend, se le puso un nombre y se establecieron los comandos de build y start, tras investigar y varios fallos, se llegó a la conclusión de que Render no soportaba el entorno de Java, que es como estaba formado gran parte del proyecto, por ello se optó por crear un dockerfile personaliza en el proyecto dentro de la carpeta del backend como podemos ver a continuación:

```
eventos > Dockerfile
1  # Imagen base con Java 17
2  FROM eclipse-temurin:17-jdk-alpine
3
4  # Establecer el directorio de trabajo
5  WORKDIR /app
6
7  # Copiar el wrapper de Maven y el resto del proyecto
8  COPY . .
9
10 # Dar permisos de ejecución al wrapper
11 RUN chmod +x mvnw
12
13 # Construir el proyecto (instala dependencias y compila)
14 RUN ./mvnw clean install -DskipTests
15
16 # Exponer el puerto (ajústalo si tu app usa otro)
17 EXPOSE 8080
18
19 # Comando por defecto para ejecutar el .jar
20 CMD ["java", "-jar", "target/eventos-0.0.1-SNAPSHOT.jar"]
```

Código 16: Dockerfile para conectar con Render

Con esto ya estará el backend disponible y listo para usarse, lo siguiente fue crear la base de datos dentro de Render que ofrece la posibilidad de crear una base de datos PostgreSQL al igual que la usada

en el entorno local. Por lo tanto se ha creado la base de datos asignando un nombre, un usuario y contraseña, además de unas URLs externas e internas.

Para poder conectarlo con el backend ya creado y funcione correctamente, dentro del archivo `application.properties` que se encuentra en el backend se han cambiado las direcciones que apuntaban hacia url, name y password de la base de datos local a la creada en el servidor Render, además en el backend del servidor se añadieron tres Environment Variables con la URL, USERNAME y PASSWORD de la base de datos creada. Con esto ya solo faltaría la última parte que es el frontend y conectarlo todo.

Para el frontend se ha creado un nuevo "Static Site" que apuntará a la carpeta de frontend y establecido un build command para su arranque, con esto ya tendremos los tres puntos creados y desplegados en el servidor funcionando correctamente como podemos ver:

SERVICE NAME 3	STATUS	RUNTIME	REGION	DEPLOYED ↓	
 tfg-frontend	✓ Deployed	Static	Global	51min	...
 tfg-eventos	✓ Deployed	Docker	Frankfurt	52min	...
 eventos	✓ Available	PostgreSQL 16	Frankfurt	1d	...

Ilustración 12: Servicios creados en Render

Sin embargo, aún falta un detalle para que se pueda usar correctamente y es cambiar las peticiones fetch para que apunten al backend de render y no al de mi servidor local, para ello lo primero ha sido crear un archivo `.env` en el frontend que contiene :

```
frontend >  .env
1  REACT_APP_BACKEND_URL=https://tfg-eventos.onrender.com
2  |
```

Código 17: URL para detectar Backend

Esto será una variable que contendrá un enlace al backend del servidor, el siguiente paso será ir a todos los archivos del frontend que hacen solicitudes fetch y añadirles al principio esta línea que conectará con el archivo `.env` y la dirección al backend del servidor.

```
const BACKEND_URL = process.env.REACT_APP_BACKEND_URL;
```

Código 18: Variable con enlace a Backend

Finalmente habrá que cambiar las peticiones fetch de como estaban y podíamos ver en el apartado anterior a la siguiente forma, para no ser repetitivo se va a poner de ejemplo cómo cambia la petición fetch de LoginPage.js siendo muy similar para el resto:

Antes:

```
✓ const res = await fetch("http://localhost:8080/api/usuarios/login", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify(formData)  
});  
  
✓ if (res.ok) {  
  const data = await res.json();  
  localStorage.setItem("usuario", data.nombreUsuario);  
  localStorage.setItem("usuarioId", data.id);  
  navigate("/");  
} else {  
  const errorMsg = await res.text();  
  alert(errorMsg);  
}  
};
```

Código 19: Conexión Backend y Frontend en entorno local

Ahora:


```

    try {
      const res = await fetch(`${BACKEND_URL}/api/usuarios/login`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(formData),
      });

      if (res.ok) {
        const data = await res.json();
        localStorage.setItem("usuario", data.nombreUsuario);
        localStorage.setItem("usuarioId", data.id);
        navigate("/");
      } else {
        const errorMsg = await res.text();
        alert(errorMsg);
      }
    } catch (error) {
      alert("Error al conectar con el servidor.");
      console.error(error);
    }
  });
}

```

Código 20: Conexión Backend y Frontend en el servidor

Como se puede apreciar simplemente cambia la dirección de localhost:8080 por la URL establecidas en .env al backend del servidor, habrá que hacer esto en todas las peticiones fetch y finalmente ya estaría disponible el servidor para su uso, a través del siguiente enlace:

<https://tfg-frontend-9xm5.onrender.com>

Hay que añadir que la base de datos sólo ofrecía un mes gratuito por lo que se pasó a una de pago y así poder tenerla de forma permanente, al igual que con el backend donde tras varios problemas con algunos navegadores y problemas con la inactividad, también se ha optado por pasarlo a uno de pago.

Punto 7: Pruebas y resultados

En este apartado se detallan las pruebas realizadas al sistema una vez implementado, así como los resultados obtenidos a nivel funcional. Se describe cómo se ha verificado que cada módulo cumple con su propósito y se representan ejemplos prácticos desde la perspectiva del usuario. Las pruebas se han llevado a cabo en un entorno local, utilizando tanto el backend desplegado en Spring Boot como el frontend en React.

7.1 Registro

Tras acceder a la página arriba a la derecha en el header saldrá la opción de registrarse, la cual nos llevará a la RegisterPage.js donde aparecerá el formulario a rellenar con el nombre de usuario, correo y una contraseña, como podemos ver a continuación:

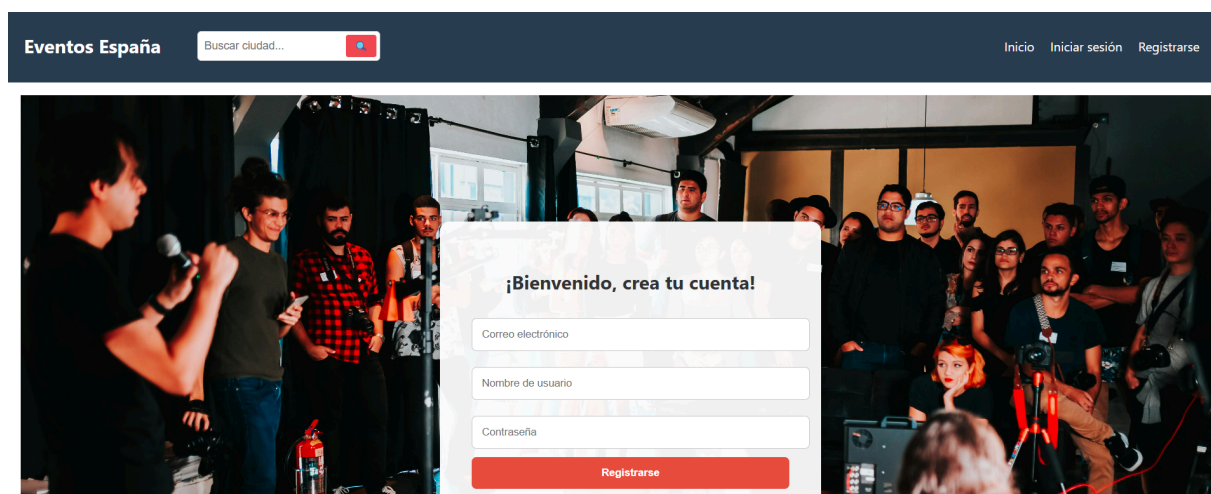


Ilustración 13: Página de Registro

Si introducimos los datos y le damos al botón de registrarse y no hay ningún fallo, el usuario quedará registrado y te llevara a la página principal con un mensaje de bienvenida y diferentes opciones nuevas



Ilustración 14: Header tras Iniciar Sesión

7.2 Inicio de sesión

Al lado de la opción de registrarse tendremos la opción de iniciar sesión, la cual al pulsar en ella nos llevará a LoginPage.js donde encontraremos un formulario mediante el cual con el correo y contraseña que se introdujeron durante el periodo de registro nos permitirá iniciar sesión, si no hay ningún error mostrará el nombre de usuario junto a un mensaje de bienvenida.

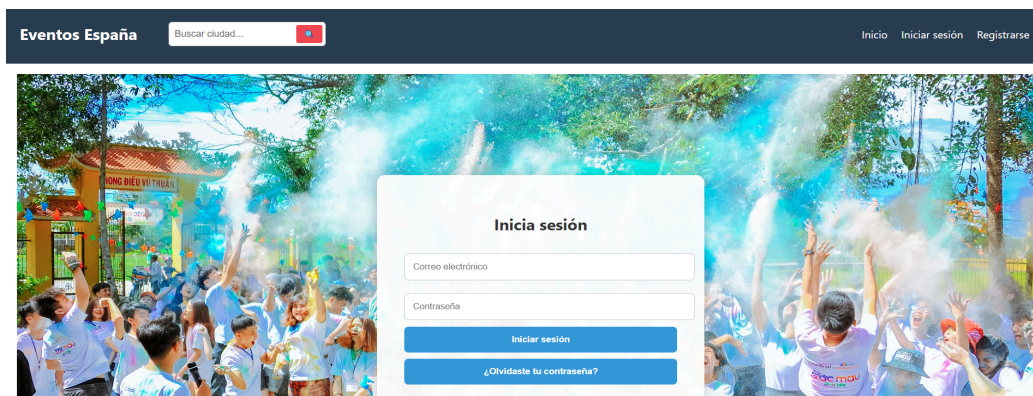


Ilustración 15: Página de iniciar Sesión

Una vez iniciada la sesión, al pulsar sobre bienvenido y nombre de usuario, saldrá un menu desplegable con dos opciones siendo una de ellas la de cerrar sesión, donde tras pulsar se cerrará la sesión activa y estaremos en la página de inicio con las opciones de inicio de sesión y registro disponibles.

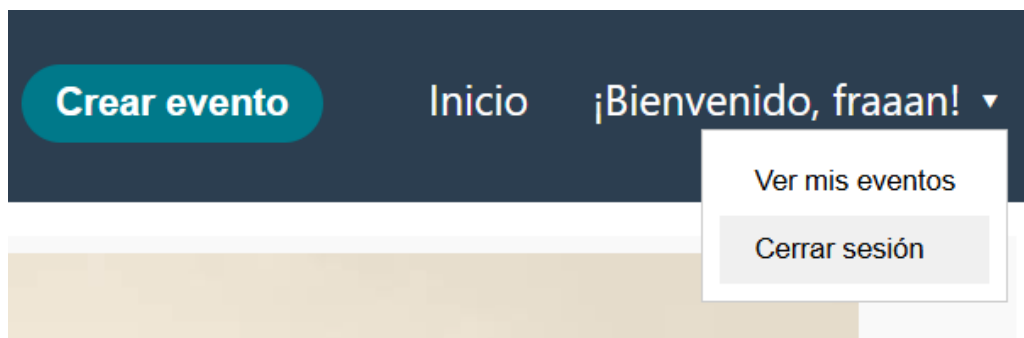


Ilustración 16: Menú de opciones de usuario

7.3 Cambiar contraseña

En la ilustración 15, si pulsamos sobre cambiar contraseña, esto nos llevará a un nuevo menú donde se nos pedirá introducir el correo electrónico donde queremos que se envíe el código de recuperación de contraseña, tras introducirlo nos pedirá que pongamos el código que nos ha llegado al mail, y tras comprobarlo y ver si es correcto, nos dejará poner la nueva contraseña dos veces para confirmar y así poder cambiarla.



Ilustración 17: Menú para introducir código para el cambio de contraseña

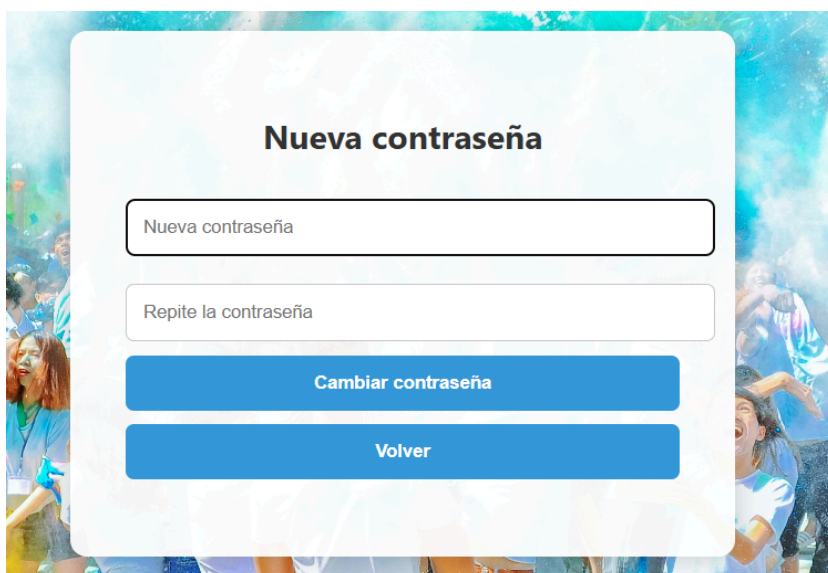


Ilustración 18: Menú de cambio de contraseña

Una vez que hemos iniciado sesión como hemos visto antes aparecerá la opción de crear evento junto al mensaje de bienvenida y al pulsar nos llevará a la página de CrearEvento.js donde tendremos un formulario a rellenar con los diferentes campos del evento como son el nombre, descripción, categoría, fecha, precio, capacidad, ubicación y una url a la imagen, si no ha habido ningún problema nos saldrá un mensaje de confirmación y el evento se creará correctamente, como podemos ver en la imagen de a continuación.



Ilustración 19: Creación de evento

7.5 Visualizar eventos

En nuestra página principal, tras un mensaje inicial con una foto, tendremos que se ha dividido en dos secciones, a la izquierda habrá una columna con diferentes filtros que se tratará más adelante, y a la derecha parecerán todos los eventos disponibles en el futuro en nuestra página, si no se aplica ningún tipo de filtro estos aparecerán, uno tras otro en filas de tres eventos, donde podremos ver una foto, el nombre del evento, la categoría a la que pertenece de las establecidas y la fecha en la que tendrá lugar, además tendrá un botón de más información que nos llevará a la página propia del evento, con la foto en grande a la izquierda y a la derecha toda la información desde el creador con su valoración, una descripción más detallada y la opción de inscribirse si queda hueco.



Cerrar sesión

Comida nazaí en la Alhambra

Categoría: Gastronomía

Descripción: Se realizará una comida en los jardines de la Alhambra basada en un menú de comida nazarí. Se abrirán la puertas a las 21:00 en la calle Real de la Alhambra para todo aquel que desee disfrutar de este menú y conocer gente y la historia de Granada. Cualquier alergia consultar a los camareros.

Fecha: 2025-10-18

Precio: 33€

Capacidad: 30

Ubicación: Granada

Organizado por: prueba con valoración media de **8.0/10**, con **2** eventos valorados

Apuntarse

Ilustración 20: Información Evento

7.6 Inscribirse

Al darle a más información del evento, esto nos lleva a la página del mismo con la opción de inscribirse en él, hay varias posibilidades, una de ellas es que hayamos accedido a la información del evento sin estar registrado, luego en lugar de inscribirse pondrá el siguiente mensaje:

 Inicia sesión para apuntarte a este evento.

Ilustración 21: Mensaje en eventos si no estás registrado

Otra opción es que la capacidad del evento sea ya de 0 y no queden huecos disponibles, lo que aparecerá lo siguiente:

Capacidad: 0

Ubicación: Granada

Organizado por: fraaan con valoración media de **7.0/10**, co

 Este evento ya no tiene plazas disponibles.

Ilustración 22: Mensaje eventos completos

La ultima opciones que el usuario si este registrado y queden huecos disponibles, luego aparecerá el botón de inscribirse normal y se podrá apuntar sin problema, una vez inscrito aparecerá un mensaje confirmando que estas inscrito ya en ese evento:

 Ya estás inscrito en este evento

Ilustración 23: Mensaje tras inscribirse

7.7 Filtros y buscador

Aquí se han añadido dos funcionalidades la primera de ellas es un buscador por ciudad, en el header aparecerá un buscador donde se podrá introducir la ciudad donde queremos buscar los eventos, tras escribir la ciudad y darle a buscar, los eventos que aparecerán serán solo de la ciudad deseada, se puede ir cambiando de ciudad en cualquier momento, incluso dejar el campo vacío y aparecerán los eventos de todas las ciudades, aquí podemos ver los evento que tendrán lugar en Granada.

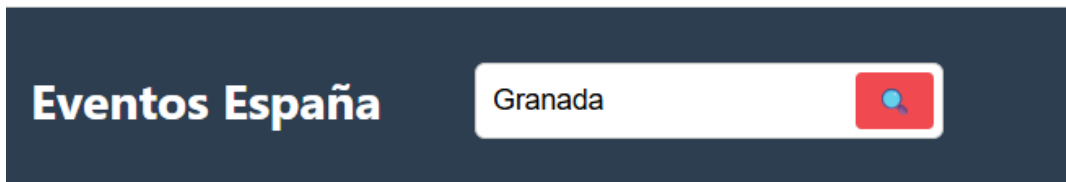


Ilustración 24: Buscador de ciudad en el header

Eventos disponibles en Granada

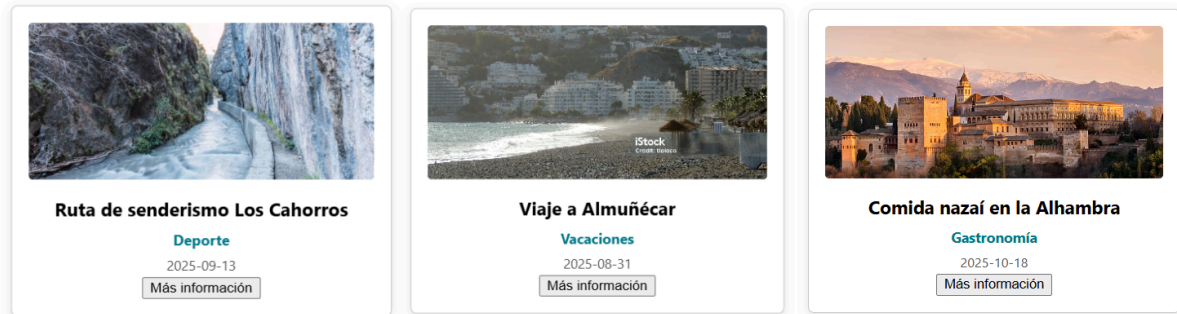


Ilustración 25: Resultados búsqueda Eventos en Granada

La otra opción de filtrado estará en la página principal en una columna a la izquierda donde aparecerán filtros fijos según la categoría, precio y fecha, podremos seleccionar una opción de cada apartado, al seleccionarla se rellenará con un recuadro de color y debajo aparecerá un botón de buscar que nos mostrará los eventos según los filtros que hayamos seleccionado, también habrá una opción de restablecer filtros, que quitará cualquier filtro y mostrará los eventos disponibles y ya.

Buscar por categoría

Música

Deporte

Vida Nocturna

Vacaciones

Videojuegos

Gastronomía

Negocios

Quedadas

Online

Otras

Buscar por precio

Gratis

1-10€

10-25€

25-50€

50-100€

Más de 100€

Buscar por fecha

Hoy

Mañana

Esta semana

Este mes

Dentro de un mes

Buscar

Restablecer filtros

Ilustración 26: Filtros de búsqueda menú izquierdo.

7.8 Gestión de eventos creados

Como se comentó antes, una vez iniciado sesión, y darle a bienvenido a parte de la opción de cerrar sesión tenemos la de ver mis eventos, que nos lleva a otra página la cual se divide en dos columnas, a la izquierda tenemos los eventos inscritos que se verá en el siguiente apartado y a la derecha los eventos creados por mí, estos eventos aparecerán en filas de dos eventos y tendrán tres opciones, una es la de modificar evento que nos llevará a una página similar a la de crear evento y podremos editar cualquier campo del mismo, otra es la de eliminar evento, donde podremos eliminar definitivamente el evento

creado y la última es la de ver inscritos, que al pulsar en ella se desplegará una lista con los usuarios inscritos en mi evento, como podemos ver a continuación:

Crear evento

Inicio

¡Bienvenido, fraaan! ▾

Ver mis eventos

Cerrar sesión

Eventos que he creado



Ruta de senderismo Los Cahorros

2025-09-13

Deporte

Modificar

Eliminar

Ver inscritos

Usuarios inscritos:

- paco123
- manolo123



Viaje a Almuñécar

2025-08-31

Vacaciones

Modificar

Eliminar

Ver inscritos

Activar Windows

Ilustración 27: Mis eventos creados

7.9 Gestión de eventos inscrito

La columna de la izquierda de mis eventos tendrá como hemos comentado anteriormente los eventos a los que me he inscrito, y habrá dos posibilidades, si el evento no ha ocurrido todavía aparecerá la opción de desapuntarse del mismo, aumentando en uno la capacidad del mismo, y la otra opción sería si un evento que has participado ya ha finalizado, aparecerá la opción de valorarlo, dejando una nota entre 1 y 10 y debajo aparecerá la media de las valoraciones de todos los usuarios.

Eventos en los que estoy inscrito



Fiesta de disfraces

2025-07-26

Vida Nocturna

Desapuntarse



Fiesta de rock

2025-07-10

Vida Nocturna

Valorar

Valoración media actual : **7.5/10** (2 valoraciones)

Ilustración 28: Mis eventos inscrito

7.10 Valoración de eventos

Una vez finalizado un evento, se podrá dejar una valoración entre el 1 y el 10 a este evento, apareciendo un mensaje de confirmación si es la primera vez que lo valoras y uno de error si ya lo has valorado, esta valoración se guardará en el evento y hará media con las de los demás, haciendo que un usuario tenga un media global de todas las medias de sus eventos, y que se muestre cuando le das a la información de un evento.

7.11 Comentar eventos

Al igual que dejar una valoración, también se dispone de la opción de dejar un comentario al evento, tras el botón de valorar, aparecerá un botón de comentar donde aparecerá un recuadro para dejar un comentario el cual se guardará en la tabla inscripción entre el usuario y el evento, además debajo de la nota media del evento tendremos un botón desplegable con los comentarios sobre el evento con el nombre del usuario que lo ha realizado como podemos ver a continuación:



Ilustración 29: Comentar evento

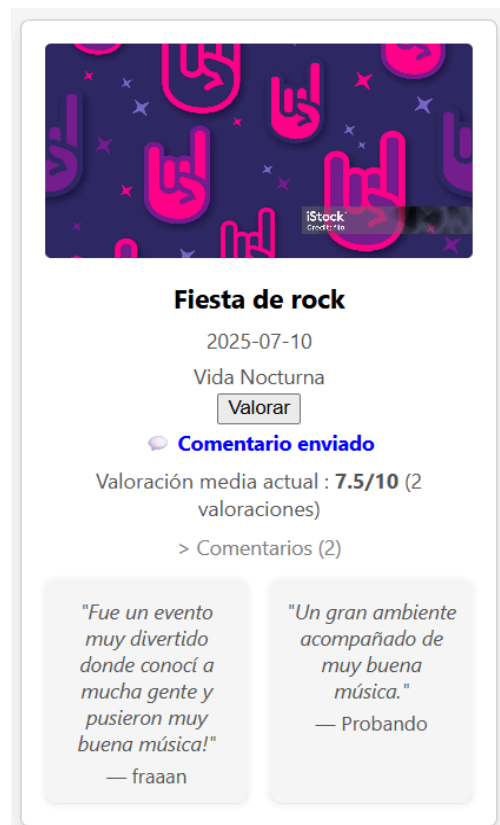


Ilustración 30: Evento comentado

Despliegue en servidor

Una vez finalizado el desarrollo y desplegado correctamente el proyecto en el servicio Render, la aplicación web quedó accesible públicamente a través de un enlace web. Esto permitió que diferentes usuarios externos —principalmente amigos y familiares— accedieron a la plataforma desde sus dispositivos móviles y ordenadores para probar su funcionamiento en condiciones reales.

Los usuarios realizaron acciones como registrarse, iniciar sesión, crear nuevos eventos, apuntarse a otros ya existentes y explorar las funcionalidades de búsqueda. En general, la experiencia fue positiva: la interfaz fue considerada intuitiva, y las funcionalidades respondieron adecuadamente.

Uno de los comentarios más repetidos fue la claridad del diseño y la facilidad con la que se podían apuntar a eventos. También se señaló que la carga inicial del sitio podría tardar unos segundos en

algunos momentos, lo cual se debe al funcionamiento del servidor gratuito de Render, que entra en modo reposo tras un periodo de inactividad. Esto fue considerado comprensible dada la naturaleza gratuita del servicio.

Esta prueba real permitió validar que el sistema es completamente funcional de cara al público y ofrece una buena experiencia de usuario. Además, permitió recoger sugerencias que podrían implementarse en futuras versiones, como mejoras visuales o la posibilidad de comentar eventos directamente desde el listado.

Punto 8: Conclusiones y trabajo futuro

8.1 Conclusiones

Este Trabajo de Fin de Grado ha consistido en el desarrollo de una plataforma web para la creación y descubrimiento de eventos, pensada como una solución ante la escasa oferta de herramientas que permitan la autogestión abierta y descentralizada de este tipo de actividades. Durante el proyecto se han abordado las fases de ciclo de vida de un desarrollo web completo, como son el análisis, diseño, implementación, pruebas y despliegue, donde cada etapa ha contribuido a mi desarrollo profesional, ofreciendo unos aprendizajes muy valiosos.

La plataforma permite a cualquier usuario, sin conocimientos técnicos, acceder a eventos según su ubicación e intereses y también crear sus propios eventos. La aplicación está construida con una arquitectura basada en React para el frontend, Spring Boot para el backend y PostgreSQL como sistema de bases de datos. Todo el sistema ha sido desplegado en la nube usando Render, garantizando así su accesibilidad desde cualquier dispositivo con conexión a Internet. Lo principal para valorar en este proyecto es la gran cantidad de conocimientos adquiridos, ya que se ha trabajado con un sistema completo donde se ha desarrollado tanto el backend como el frontend ofreciendo una gran oportunidad para aplicar los conocimientos adquiridos durante los 4 años del grado en Ingeniería Informática y para sobre todo aprender, especialmente en lo relacionado con el desarrollo web, programación orientada a objetos, bases de datos, seguridad y despliegue de aplicaciones.

El hecho de usar tecnologías actuales y de las más utilizadas ha sido un gran punto a favor, ya que estaba bastante interesado en JavaScript debido a que es un lenguaje bastante utilizado e interesante.

Durante la realización del proyecto se ha seguido un enfoque iterativo, corrigiendo errores durante la implementación y realizando pruebas funcionales en todo momento, además se ha ido siguiendo la lista de objetivos planteada en el primer apartado.

Una de las mayores satisfacciones ha sido comprobar que el sistema funciona correctamente en un entorno real, con usuarios externos accediendo a la plataforma realizando acciones clave como inscribirse, valorar, comentar eventos. Esto refuerza la utilidad del proyecto no solo como ejercicio académico, sino también como producto potencial para su uso en entornos locales o comunitarios.

8.2 Trabajo Futuro

El proyecto actual a pesar de ser completo, puede ser el comienzo de algo mayor y más ambicioso, durante su desarrollo han surgido diferentes funcionalidades las cuales no se han añadido por varios motivos, pero no está mal señalarlas.

Una posible funcionalidad que se comentó pero finalmente no se añadió es la opción de añadir métodos de pago al sistema, ya que como se menciona en puntos anteriores si un evento es de pago, habrá que pagar en el momento en el que se realiza el evento de manera física, ya que la aplicación es solo para inscribirte y ver las personas que irán al evento, luego se podrían añadir métodos de pago como con tarjeta de crédito, por paypal, bizum o transferencia, siendo necesarias más medidas de seguridad y protección de datos.

Otra posible mejora puede ser un sistema de notificaciones, donde ya sea por correo electrónico o bien a través de la misma página, se envíen notificaciones de que te has inscrito, si el evento ha cambiado o si te han dejado una valoración o comentario si eres un creador de un evento finalizado. También se podría añadir un sistema de chat para que los usuarios puedan escribirse con el creador de los eventos y consultar dudas, cambio o sugerencias para los eventos, incluso uno con una IA o moderador del sistema que ayude en todo momento.

A parte de estas funcionalidades concretas también se podrían añadir mejoras visuales como animaciones, transiciones, incluso convertir la plataforma en una aplicación progresiva (PWA) o app nativa para facilitar su uso desde el móvil. Si se quisiera convertir en un proyecto totalmente real, se podría migrar a un servidor de pago y reforzar la seguridad, mediante cifrado de contraseñas, control de errores y uso de HTTPS bajo dominio propio.

Bibliografía

- [1] Escuela Internacional de Protocolos y eventos: Sabías qué... ¿Cuál fue el primer evento de la historia? Accedido 2 de Julio de 2025 de <https://eipgranada.com/sabias-que-cual-fue-el-primer-evento-de-la-historia/>
- [2] Marcela Argumedo de Erice & Marcelo di Cesar (2013). Cátedra Organización de Eventos.
- [3] Eventbrite. Sitio Web Oficial Accedido 4 de Julio de 2025 de <https://www.eventbrite.es/>
- [4] Meetup. Sitio Web Oficial Accedido 4 de Julio de 2025 de <https://www.meetup.com/home/?suggested=true&source=EVENTS>
- [5] Facebook Events. Sitio Web Oficial Accedido 4 de Julio de 2025 de <https://www.facebook.com/events/>
- [6] Granada Social. Sitio Web Oficial Accedido 4 de Julio de 2025 de <https://granadasocial.org/>
- [7] Interjet. ¿Que es la ley de protección de datos?. Accedido el 5 de Julio de 2025 de <https://ingertec.com/compliance-2/ley-proteccion-datos/>
- [8] Parlamento Europeo. El Presidente M.Schulz & Consejo. La Presidenta J.A. Hennis-Plasschaert (2016). REGLAMENTO (UE) 2016/ 679 DEL PARLAMENTO EUROPEO Y DEL CONSEJO
- [9] Juan Diego Gauchat. (2012). El gran libro de html5, css3 y javascript.
- [10] Escuela Británica de Artes y Tecnologías: Que es React y para qué sirve. Accedido el 10 de Julio de 2025 de <https://ebac.mx/blog/que-es-react>
- [11] React. Accedido 10 de Julio de 2025 de <https://es.react.dev/>
- [12] Tokyo School: ¿Qué es Spring Boot y para qué sirve?. Accedido el 11 de Julio de 2025 de <https://www.tokioschool.com/noticias/spring-boot/>
- [13] Spring. Spring Data JPA. Accedido el 11 de Julio de 2025 de <https://spring.io/projects/spring-data-jpa>
- [14] Ayudaley: Qué es PostgreSQL y sus principales ventajas. Accedido el 12 de Julio de <https://ayudaleyprotecciondatos.es/bases-de-datos/que-es-postgresql-ventajas/>
- [15] Render . Documentación oficial. Accedido el 28 de Julio de 2025 de <https://render.com/docs>